基于几何视域遮蔽与智能优化的干扰弹协同投放策略

摘要

烟幕干扰技术作为一种高效且低成本的对抗手段，在现代战争中具有重要作用。该技术通过化学燃烧或爆炸生成烟幕或气溶胶云团，在目标区域形成遮蔽效应，以干扰敌方导弹的视线，从而保护高价值目标。随着无人机技术的迅速发展，利用无人机进行烟幕干扰弹的投放与引爆，已成为提升干扰效能的关键途径。本文针对空地导弹来袭场景，分别对单无人机单弹、单无人机多弹、多无人机单弹以及多无人机多弹四种烟幕弹投放策略的优化问题展开研究，旨在通过精确调控无人机的飞行参数（包括航向与速度）与干扰弹的投放参数（如投放时间和起爆时间），最大化对目标的遮蔽持续时间，进而显著提升防御效能。

针对问题一, 构建通用**运动轨迹模型**、**真目标参数化模型**及**三维视域遮蔽分析模型**三个核心模型，探究单无人机单次投放烟幕干扰弹的防御效果。仿真结果表明：该策略可达成 1.405 秒的连续有效全遮蔽，遮蔽时间窗口为起爆后 2.943 秒至 4.348 秒。

针对问题二, 构建了以最大化有效遮蔽时长为目标的策略优化模型，探究单无人机单次投放烟幕干扰弹的投放策略，提出**双阶段混合鲸鱼优化算法（TSH-WOA）**，融合**全局粗筛**与**局部精调**的混合搜索策略，并引入自适应**高斯变异机制**。在严格满足物理约束条件下，最终实现了长达 4.580 秒的有效遮蔽时长。

针对问题三, 构建单无人机多弹的协同遮蔽优化模型，引入容斥原理量化联合遮蔽效果，设计了基于软化逻辑与重参数化的约束处理机制，构建**分层优化**框架，采用**三段式混合鲸鱼优化算法**，三枚干扰弹分别贡献 4.58 秒、4.24 秒和 4.02 秒的单体遮蔽时间，最后通过剔除重合部分，得到最终优化结果为 6.74 秒。

针对问题四, 构建多无人机单弹的协同任务分配与时空资源调度模型，提出基于**骨架反演**，**贪心搜索**与**束搜索混合**的协同优化算法。通过离散候选池构造与组合优化，高效制定投放策略。最终实现了 8.760 秒的有效遮蔽时长，三架无人机分别贡献 2.930 秒、3.680 秒和 2.155 秒的单体遮蔽时间，通过协同配合实现了遮蔽时段的有效衔接与互补。

针对问题五, 构建多导弹-多无人机-多干扰弹分层任务分配与资源调度优化模型，结合**边际增益算法**与时空覆盖优化理论，通过离散候选事件池构造、战略层无人机-导弹任务指派、以及战术层时空资源调度，最终得出最优协同策略，实现了 19.852 秒的总有效遮蔽时长。

关键字：多体运动仿真　鲸鱼优化算法　非凸组合优化　蒙特卡洛采样　时空离散

一、 问题重述

本问题旨在研究聚焦于无人机烟幕干扰弹的投放策略问题，包括无人机的飞行参数（包括航向与速度）以及干扰弹的投放参数（如投放时间和起爆时间）。来袭导弹飞行速度已知，初始坐标已知，其飞行方向指向一个可建模为坐标原点的假目标。真目标是底面半径、圆心与高度皆已知的圆柱体。执行干扰任务的五架无人机初始位置均已知，在 70-140 m/s 的速度范围内，以恒定高度进行匀速直线飞行，飞行方向与速度一经确定即不可更改。烟幕干扰弹投放后，在重力作用下作抛物线运动。起爆后球形云团匀速下降。有效遮蔽的判定条件是在可提供有效遮蔽的 20s 内，导弹与真目标上任意一点的连线都必须穿过球形云团。

问题一： 无人机 FY1 以 120 米/秒的速度保持恒定高度作匀速直线飞行。设定烟幕干扰弹的投放时间点 1.5s，仅投放 1 枚烟幕干扰弹。该枚干扰弹于投放后 3.6 秒起爆，云团起爆后的速度匀速下沉。需要求解从烟幕干扰弹起爆时刻开始直到失效的时间内，云团能够有效遮蔽导弹 M1 视线的总时长。

问题二： 问题二需要设计**单无人机单干扰弹**的投放策略，包括无人机的飞行参数（飞行速度、方向等）以及干扰弹的投放参数（投放位置、爆炸位置）。目标是在题目给出的约束条件下，最大化云团对导弹 M1 的有效遮蔽时长，确保真目标得到充分保护。

问题三： 问题三需要设计**单无人机多干扰弹**的投放策略。确保每两枚干扰弹的投放时间间隔至少为 1 秒，并且考虑单个干扰弹以及多个干扰弹联合遮挡的情况，目标是合理规划三枚干扰弹的投放与起爆时序，使得云团对 M1 导弹的有效遮蔽时间尽可能延长，遮蔽时段可以不连续。结果保存至 result1.xlsx。

问题四： 问题四需要设计**多无人机单干扰弹**的投放策略。3 台无人机同时开始飞行，目标是合理规划 3 个无人机的无人机的飞行参数以及干扰弹的投放参数。结果保存至 result2.xlsx。

问题五： 问题五需要设计**多无人机多干扰弹**的投放策略。5 台无人机同时开始飞行，中途至多投放 3 个烟幕干扰弹，目标是合理规划 5 个无人机的飞行方向、飞行速度、烟幕干扰弹的投放点以及起爆点。结果保存至 result3.xlsx。

二、 问题分析

根据有效遮蔽的定义，要求导弹与真目标之间的任意一条视线都被烟幕干扰弹爆炸后形成的云团阻断。当云团在垂直于该视线方向上的截距长度小于其半径时，达到遮蔽效果。

首先，建立以假目标位置为原点的三维空间坐标系，对无人机及导弹的运动过程进行向量化建模与仿真。建立无人机、导弹、烟幕干扰弹在投放后，以及干扰弹起爆后形成云团的下沉过程的通用运动轨迹方程。

2.1 问题一分析

在问题一中，基于通用运动轨迹方程，对圆柱体真目标进行参数化建模与曲面重建。在此基础上，构建三维空间遮挡动态检测与视域分析模型，在离散时间步下通过计算导弹视线与烟幕云团之间的最短距离，动态判断所有样本点是否均被遮蔽。最终，依据遮蔽状态矩阵中从全部遮蔽到首次出现未遮蔽点的时间区间，确定有效遮蔽时长。

2.2 问题二分析

在问题二中，基于问题一建立的真目标参数化与曲面重建模型以及三维空间遮挡动态检测与视域分析模型，根据通用运动轨迹方程，针对单无人机投放多枚烟幕弹的任务场景，建立以最大有效遮蔽时长为优化目标的投放策略优化模型，协同调整无人机的航向角（0°360°）、飞行速度（70m/s140m/s）与飞行参数。为提升求解效率与质量，需要对现有的优化算法进行改进，制定高效投放策略。

2.3 问题三分析

在问题三中，针对单无人机投放三枚烟幕弹的协同遮蔽场景，构建一个以最大化联合有效遮蔽时长为目标的优化模型，需要协同优化无人机的飞行参数，并考虑单弹独立遮蔽与多弹协同遮蔽两种作用模式的综合效果。在建模过程中，需要处理无人机的速度约束、烟幕弹的有效作用时间窗口、最小投放间隔要求等约束条件。考虑到问题的高维性和非凸特性，采用分阶段优化的策略，并使用全局优化算法进行粗搜索，再结合局部优化方法进行精细调优，通过基于软化逻辑的约束处理技术进行逐步探索。

2.4 问题四分析

在问题四中，首先需要明确三架无人机的初始位置和可达范围，以及导弹 M1 的飞行轨迹和时间窗口。由于每架无人机仅投放一枚烟幕弹，需要综合考虑飞行参数和约束条件，使三枚烟幕弹的联合遮蔽时间尽可能长。同时，为了保证物理可行性，需要引入航向正则和个体有效时间约束。最终，通过离散候选池构造与组合搜索的方法，选取最优的投放时刻和位置策略。

2.5 问题五分析

在问题五中，针对五架无人机协同干扰三枚导弹的复杂问题，需建立分层优化框架：首先根据无人机与导弹的相对时空关系进行任务分配，确定每架无人机主要负责干扰的导弹目标；接着为每架无人机规划可行的飞行航向与速度组合，生成满足物理约束的候选起爆事件集合；然后基于时空覆盖优化理论，以最大化最短遮蔽时间为优先目标，通过边际增益算法协调多机投放时序，避免资源重叠的同时确保遮蔽连续性；最后综合考虑投放间隔约束和弹舱容量限制，得出完整策略方案。

三、 模型假设

为了模型更加合理简化问题，本文建立以下假设

- 忽略空气阻力作用
- 采用右手直角坐标系，单位 m、s，重力沿 $-z$
- 时间起点 $t = 0$ 为任务开始
- 真目标为实体圆柱，顶/底面与侧面均参与遮蔽判定
- 地面为 x-y 平面，$z = 0$，不考虑地形起伏
- 相交判据容差 $R \sim 10^{-6}m$；完全遮蔽容差为“被遮蔽点比例 $\geq 99\%$”以抗采样漏判
- 若导弹抵达目标前即被击中或超出场景，终止仿真且不计后续时长

四、 符号说明

符号	说明	单位
P(研究对象, t)	t 时刻研究对象的位置向量	m
P_x(研究对象, t)	t 时刻研究对象的 y 坐标值	m
P_y(研究对象, t)	t 时刻研究对象的 y 坐标值	m
P_z(研究对象, t)	t 时刻研究对象的 z 坐标值	m
d(研究对象, t)	t 时刻研究对象的方向向量	—
u(研究对象, t)	t 时刻研究对象的单位方向向量	—
v(研究对象, t)	t 时刻研究对象的速度值	m/s
M_n	第 n 颗空地导弹	—
FY_n	第 n 个无人机编号	—
t_{Drop}	烟幕干扰弹的投放时间点	s
t_{Boom}	干扰弹的起爆时刻	s
$t_{\text{云团}}$	云团的运动时间（自干扰弹起爆起算）	s
θ	以 x 轴为正向的偏转角，取值区间 $[0, 360]$	°
(H, R)	圆柱体的高与底面半径	m
L(目标 1, 目标 2, t)	在 t 时刻下，目标 1 面向目标 2 的视线向量	m
d_{min}	云团中心到其在视线向量投影点的欧几里得距离	m
N_i	在圆柱体目标对象上取样点的点云集合	—
$M_{\text{遮蔽},t}[h]_{N\times 1}$	在 t 时刻下，描述取样点视线遮挡状况的遮蔽矩阵	—
SJ_i	第 i 个烟幕干扰弹	—

五、 通用运动轨迹模型

对于从无人机受领任务时刻 ($t_0 = 0$) 开始，构建物体位置关于时间运动轨迹以及位置的时序模型，并以假目标为圆心建立三维空间的笛卡尔坐标系（如图1）

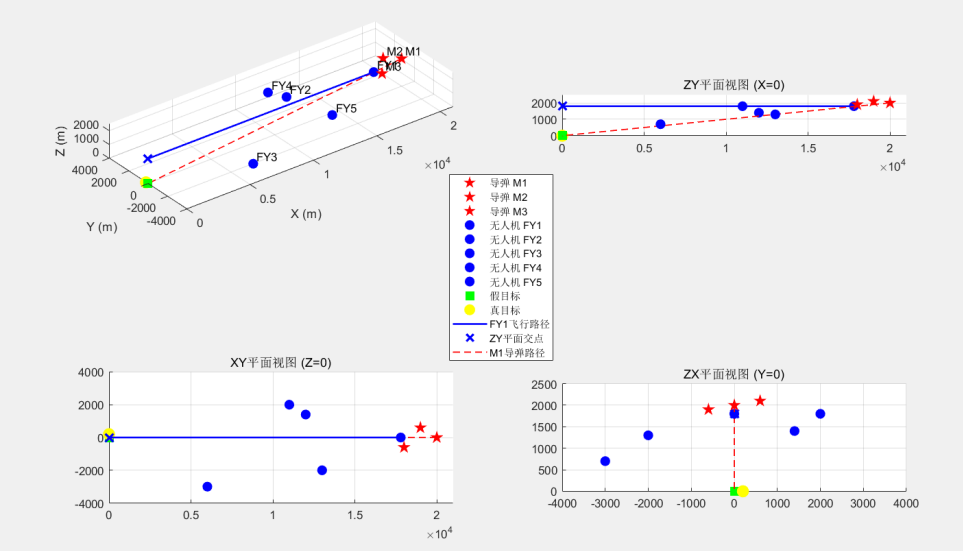


图 1　导弹–无人机–目标空间关系三维与投影可视化图

在此坐标系下，对问题一的各实体（即 FY1 无人机、M1 导弹及烟幕弹）的运动学轨迹进行三**维空间的时序建模**，并精确模拟其动态行为。通过此仿真，最终量化烟幕对导弹的有效遮蔽持续时间。

5.1 导弹的运动轨迹方程

对于整个过程来说，导弹的运动轨迹方程始终不变。导弹 M_n 的初始坐标 $P(M_n, t = 0) = (P_x(M_n, t = 0), P_y(M_n, t = 0), P_z(M_n, t = 0))$，可以求出运动方向向量和单位方向向量如下：

$$\mathbf{d}_{(M_n,t)} = (P_x(M_n, t = 0),\ 0,\ P_z(M_n, t = 0)) \tag{1}$$

$$\mathbf{u}_{(M_n,t)} = \left(\frac{P_x(MM_n, t = 0)}{\|\mathbf{d}_{(M_n,t=0)}\|}, 0, \frac{P_z(M_n, t = 0)}{\|\mathbf{d}_{(M_n,t=0)}\|} \right) \tag{2}$$

由题目可以得知导弹 M1 速度为 300 m/s，求出速度向量：

$$\mathbf{v}_{(M_n,t)} = 300 \cdot \mathbf{u}_{(M_n,t=0)} = \left(\frac{300P_x(M_n, t = 0)}{\|\mathbf{d}_{(M_n,t=0)}\|}, 0, \frac{300P_z(M_n, t = 0)}{\|\mathbf{d}_{(M_n,t=0)}\|} \right) \text{m/s} \tag{3}$$

导弹 M1 的运动轨迹方程如下, 无论是:

$$\begin{cases} P_x(M_n,t) = P_x(M_n,t=0) + \dfrac{300P_x(M_n,t=0)}{\|\mathbf{d}_{(M1,t=0)}\|} \cdot t \\ P_y(M_n,t) = P_y(M_n,t=0) \\ P_z(M_n,t) = P_z(M_n,t=0) + \dfrac{300P_z(M_n,t=0)}{\|\mathbf{d}_{(M_n,t=0)}\|} \cdot t \end{cases} \tag{4}$$

对于导弹 M_n 来说，设定到达原点终点的时间点为 $t_{end}(M_i)$，根据导弹的运动公式和导弹与原点的距离公式，可以求出不同导弹的

5.2 烟幕干扰弹投放时间点前无人机的运动轨迹方程

无人机烟幕干扰弹的投放时间点 t_{Drop} 是无人机从初始点 $P(\mathrm{FY}_n, t=0) = \big(P_x(\mathrm{FY}_n, t=0),\ P_y(\mathrm{FY}_n, t=0),\ P_z(\mathrm{FY}_n, t=0)\big)$ 飞向一个由角度 θ 定义的方向。

飞行方向向量: $\mathbf{d}_{(\mathrm{FY}_n,t)} = \big(\cos\theta,\ \sin\theta,\ 0\big)$

单位方向向量: $\mathbf{u}_{(\mathrm{FY}_n,t)} = \big(\cos\theta,\ \sin\theta,\ 0\big)$

速度向量: $\mathbf{v}_{(\mathrm{FY}_n,t)} = v_{\mathrm{FY}_n} \cdot \mathbf{u}_{\mathrm{FY}_n} = v_{\mathrm{FY}_n}\,(\cos\theta,\ \sin\theta,\ 0)$

无人机 FY_n 的位置方程:

$$\begin{cases} P_x(\mathrm{FY}_n,\tau) = P_x(\mathrm{FY}_n,t=0) - \dfrac{P_x(\mathrm{FY}_n,t=0)\cdot v_{(\mathrm{FY}_n)}}{\sqrt{P_x(\mathrm{FY}_n,t=0)^2 + y_0^2}} \cdot (t_1+\tau) \\ P_y(\mathrm{FY}_n,\tau) = \dfrac{y_0 \cdot v_{(\mathrm{FY}_n)}}{\sqrt{P_x(\mathrm{FY}_n,t=0)^2 + y_0^2}} \cdot (t_1+\tau) \\ P_z(\mathrm{FY}_n,\tau) = P_z(\mathrm{FY}_n,t=0) - \dfrac{1}{2}g\tau^2 \quad \text{for} \quad 0 \le \tau \le \tau_b \end{cases} \tag{5}$$

5.3 烟幕干扰弹从投放时间点到起爆时间点期间的运动轨迹方程

设起爆时间点为投放时间点之后第 t_{Boom} 秒为烟幕干扰弹起爆时间点，设烟幕干扰弹在投放后运动的时间为 τ，为起爆之后烟幕干扰弹受重力作用进行抛物线运动。

$$\begin{cases} P_x(\text{干扰弹 n，}\ \tau) = P_x(\text{干扰弹 n，}\ \tau\text{=0}) - \dfrac{P_x(\text{干扰弹 n，}\ \tau\text{=0})\cdot v_{FY_n}}{\sqrt{P_x(\text{干扰弹 n，}\ \text{t=0})^2 + y_0^2}} \cdot (t_{Drop} + (\text{干扰弹 n，}\ \tau)) \\ P_y(\text{干扰弹 n，}\ \tau) = \dfrac{y_0 \cdot v_{FY_n}}{\sqrt{P_x(\text{干扰弹 n，}\ \tau\text{=0})^2 + y_0^2}} \cdot (t_1 + (\text{干扰弹 n，}\ \tau)) \\ P_z(\text{干扰弹 n，}\ \tau) = P_z(\text{干扰弹 n，}\ \tau\text{=0}) - \dfrac{1}{2}g(\tau)^2 \end{cases} \tag{6}$$

其中，t_{Drop} 为烟幕干扰弹投放时间点。

5.4 起爆时间点后的云团的运动轨迹方程

对于云团来说，它的起始位置如下：

$$\begin{cases} P_x(\text{云团 n}, t_{\text{云团}}=0) = P_x(\text{干扰弹}_n, t=0) - \frac{P_x(\text{干扰弹}_n, t_{Boom})\cdot v_{FY_n}}{\sqrt{P_x(\text{干扰弹}_n, t_{Boom})^2+y_0^2}}(t_{Drop}+\tau_{(\text{干扰弹}_n)}) \\ P_y(\text{云团 n}, t_{\text{云团}}=0) = \frac{y_0\cdot v_{FY_n}}{\sqrt{P_x(\text{干扰弹}_n, \tau=t_{Boom})^2+y_0^2}}(t_1+t_{Boom}) \\ P_z(\text{云团 n}, t_{\text{云团}}=0) = P_z(\text{干扰弹}_n, \tau=0) - \frac{1}{2}g(t_{Boom})^2 \end{cases} \tag{7}$$

根据题目描述，云团的初始位置由给定方程确定，之后在运动过程中：x 坐标和 y 坐标保持不变（水平方向无运动），云团以 3m/s 的速度匀速下降。因此，云团 n 的运动轨迹方程可以表示为：

$$\begin{cases} P_x(\text{云团}n, t_{\text{云团}}) = x(\text{云团}n, t_{\text{云团}}=0) \\ P_y(\text{云团}n, t_{\text{云团}}) = y(\text{云团}n, t_{\text{云团}}=0) \\ P_z(\text{云团}n, t_{\text{云团}}) = z(\text{云团}n, t_{\text{云团}}=0) - 3\cdot t_{\text{云团}} \end{cases} \tag{8}$$

以上运动轨迹方程为通用公式，在其余题目中都可以运用。

六、 问题一：单无人机单次投放的有效遮蔽时长计算

6.1 基于三维向量空间的运动轨迹计算

导弹 M1 的飞行轨迹通过其初始位置及指向目标点的单位方向向量确定。将初始位置坐标与速度值代入运动学方程，可计算出导弹在任意时刻的空间位置坐标。导弹飞抵原点所需的全部飞行时间可由其初始位置模长与飞行速度的比值确定。

无人机的起爆点位置 $P(\text{干扰弹}, t=1.5+3.6s)$ 是从投弹点出发，经过 $t_{\text{Boom}}=3.6s$ 的引信延时加上下坠距离计算得到，进行物理速度模型分解，分别得到水平和竖直方向偏移量，从而得到了起爆点的新位置：

$$\begin{bmatrix} P(\text{干扰弹}, t=1.5\text{s}) + \left[d_{(\text{FY1},t)} \cdot v_{(\text{FY1},t)} \cdot \Delta t_{\text{delay}} \cdot \left(\frac{d_{(\text{FY1},t)}\cdot(-1,0,0)}{\|d_{(\text{FY1},t)}, (-1,0,0)\|} \right) \right] \\ 0 \\ \sum_{t=1.5}^{5.1} \left[v_{(\text{FY1},t)} \cdot \sqrt{1-\left(\frac{d_{(\text{FY1},t)}\cdot(-1,0,0)}{\|d_{(\text{FY1},t)}, (-1,0,0)\|} \right)^2} \cdot t - \frac{1}{2}gt^2 \right] \end{bmatrix}$$

无人机 FY1 的飞行方向单位向量记为 d_{FY}。从题目信息可以得知其初始位置为 $P(\text{FY1}, t=0)=(17800, 0, 1800)$，且飞行方向恒定，满足 $\frac{d}{dt}d_{(\text{FY1},t)}=0$，方向向量为 $d_{(\text{FY1},t)}=(0,0,0)-P(\text{FY1}, t=0)$。由于无人机保持水平飞行，速度方向不变，其纵

向偏移量为零，因此仅考虑 x 和 y 方向的分量，即 $P_z(\text{FY1}, t=0)=0$，代入通用方程可计算 FY1 飞行方向的单位向量。

无人机的投弹点位置 $P(\text{干扰弹}, t=1.5\text{s})$ 基于无人机速度 $v_{(\text{FY1},t)}$ 和第一段飞行时间 $\Delta t_{\text{drop}}=1.5\text{s}$ 计算，代入通用公式可以得出烟幕干扰弹被投放时刻 t_{Drop} 的位置。

云团中心位置的确定需先将干扰弹在烟幕干扰弹起爆点的位置坐标代入通用运动轨迹方程。云团形成后以 3m/s 的速度匀速线性下沉，其竖直坐标随时间推移均匀减小。通过云团中心的通用运动模型可求出任意时刻云团中心在三维空间中的具体位置。

6.2 三维目标表面参数化与曲面重建模型

6.2.1 圆柱体目标的参数化建模与取样

真目标柱体的中轴线与 z 轴平行，半径 $R>0$、高度 $H>0$，底面中心 $(0,200,0)$。其中，由于空地导弹视线首先接触物体的外表面，所以建模其外表面由三部分组成：

$$S = S_{\text{顶面}} \cup S_{\text{侧面}} \cup S_{\text{底面}} \tag{9}$$

6.2.1.1 真目标侧面参数化与面积要素

在侧面参数化过程中，通过映射

$$\Phi_{\text{side}} : D_{\text{side}} = [0,2\pi)\times[0,H] \to S_{\text{side}}, \quad \Phi_{\text{side}}(\theta,z) = \begin{bmatrix} R\cos\theta \\ y_0 + R\sin\theta \\ z \end{bmatrix} \tag{10}$$

将柱面侧面的几何结构表示为角度 θ 和高度 z 的函数。该映射的雅可比矩阵的两个偏导向量分别为

$$\partial_\theta \Phi = (-R\sin\theta, R\cos\theta, 0)^\top, \quad \partial_z \Phi = (0,0,1)^\top \tag{11}$$

其叉积的范数即为面积要素

$$J_{\text{side}}(\theta,z) = \|\partial_\theta \Phi \times \partial_z \Phi\| = R \tag{12}$$

因此，侧面上的面积元可表示为 $dA = R\,d\theta\,dz$，进一步可积分得到侧面总面积为

$$A_{\text{side}} = \int_{D_{\text{side}}} R\,d\theta\,dz = 2\pi RH. \tag{13}$$

若假设随机变量 (Θ, Z) 服从区间 $[0,2\pi)\times[0,H]$ 上的独立均匀分布，则通过参数化映射在侧面诱导的表面积密度为

$$g_{\text{side}}\left(\Phi_{\text{side}}(\theta,z)\right) = \frac{f_{\Theta,Z}(\theta,z)}{J_{\text{side}}(\theta,z)} = \frac{1/(2\pi H)}{R} = \frac{1}{A_{\text{side}}} \tag{14}$$

6.2.1.2 真目标顶面参数化与面积要素

对于真目标顶面，采用极坐标参数 $(r,\theta) \in D_{\text{disk}} = [0,R] \times [0,2\pi)$ 进行描述。表面的参数化映射分别定义为

$$\Phi_{\text{top}}(r,\theta) = \begin{bmatrix} r\cos\theta \\ y_0 + r\sin\theta \\ H \end{bmatrix} \quad \text{和} \quad \Phi_{\text{bot}}(r,\theta) = \begin{bmatrix} r\cos\theta \\ y_0 + r\sin\theta \\ 0 \end{bmatrix} \tag{15}$$

在圆柱体表面进行均匀采样时，需考虑其侧面、顶面与底面的总面积，并根据面积比例确定各表面的采样权重：

$$P_{\text{side}} = \frac{2\pi r H}{A_{\text{tot}}}, \quad P_{\text{top}} = P_{\text{bot}} = \frac{\pi R^2}{A_{\text{tot}}} \tag{16}$$

采样过程首先依以上权重随机选择表面类型。若选中侧面，则采样参数为

$$\Theta \sim \text{Unif}[0,2\pi), \quad Z \sim \text{Unif}[0,H], \tag{17}$$

若选中顶面或底面，则采样方式为：

$$U \sim \text{Unif}[0,1], \quad r \ldots^{=} R\sqrt{U}, \quad \theta \sim \text{Unif}[0,2\pi), \tag{18}$$

为实现顶面与底面按面积均匀采样，需使联合概率密度满足

$$f_{R,\Theta}(r,\theta) = \frac{J_{\text{disk}}(r,\theta)}{A_{\text{cap}}} = \frac{r}{\pi R^2}, \tag{19}$$

其中 Θ 服从均匀分布，半径 R^* 的边缘密度为 $f_{R^*}(r) = \frac{2r}{R^2}$。

至此，圆柱体表面已完成参数化构建，表面上任意一点 P 可由二维参数对 (θ, z) 唯一确定，且采样满足面积均匀性。具体地，当目标采样点 N_i 位于侧面时，其坐标为

$$[r\cos(\theta),\ y_0 + r\sin(\theta),\ \lambda H], \quad \lambda H \in [0,H]; \tag{20}$$

位于顶面时为 $[r\cos(\theta),\ y_0 + r\sin(\theta),\ H]$; 位于底面时则为 $[r\cos(\theta),\ y_0 + r\sin(\theta),\ 0]$。

该采样方法通过分层和混合策略，确保了圆柱体表面点的均匀分布 (如图 2)。

6.3 三维空间遮挡动态检测与视域分析模型

在物理建模中，通过分析视线与云团的空间几何关系来判断目标是否被遮蔽。本模型将目标表面离散为蒙特卡洛采样点，若任一采样点与云团中心的最小距离小于其有效半径，则该点被判定为单视线遮蔽。当所有采样点都被遮蔽时，目标对象被认为处于完全有效遮蔽状态，其总时长通过记录动态遮蔽矩阵中所有元素均为 1 的时间区间。

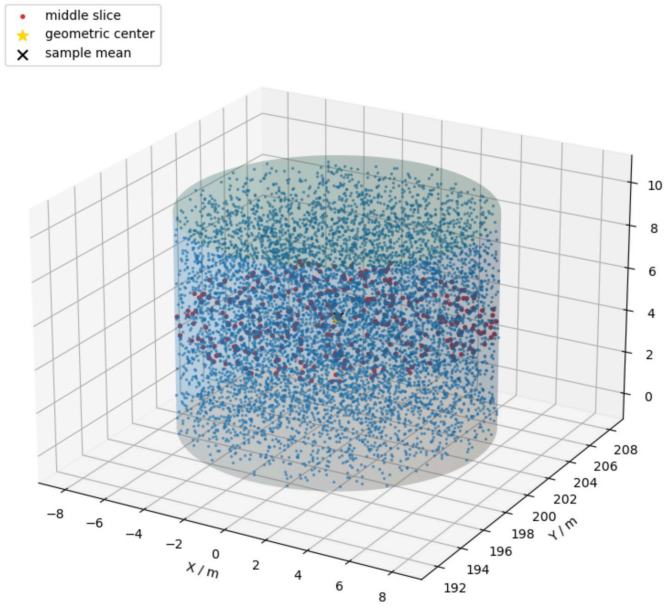


图 2　圆柱体均匀面积取样

6.3.1 视线向量构建

给定导弹 $M_{(t)}=\big(P_x(\mathrm{M1},t),\,P_y(\mathrm{M1},t),\,P_z(\mathrm{M1},t)\big)$ 和目标点 $T_i=\big(P_x(\text{假目标},t),\,P_y(\text{假目标},t),\,P_z(\text{假目标},t)\big)$，视线向量 $L\big(P(\mathrm{M1},t),(0,0,0),t\big)$ 为导弹到目标点的向量：

$$L\big(P(\mathrm{M1},t),(0,0,0),t\big)=P(\text{假目标},t)-P(\mathrm{M1},t)=\begin{bmatrix}P_x(\text{假目标},t)-P_x(\mathrm{M1},t)\\P_y(\text{假目标},t)-P_y(\mathrm{M1},t)\\P_z(\text{假目标},t)-P_z(\mathrm{M1},t)\end{bmatrix}\tag{21}$$

6.3.2 基于视线向量的遮蔽分析

为了判断云团是否遮蔽了导弹与目标之间的视线，需要利用向量投影的原理来计算云团到视线的距离。设云团中心为 $P(\text{云团中心},\mathrm{t})$，其半径为 $R_{\text{球}}$。

通过计算各个取样点 N_i 的 $P(\text{真目标取样点}N_i,t)$ 到 $P(\text{云团中心},t)$ 的最短距离，并与云团半径 $R_{\text{球}}$ 进行比较，得到“单点遮蔽情况”。

然后根据动态遮蔽矩阵，检验 $\sum_{i=1}^{N}m_{i1}=N\times1$ 来得到总体完全遮挡情况，从而可以判断离散时间 t_i 下，是否处于“有效遮挡”状态。

6.3.3 向量投影系数的几何计算

首先，计算云团中心 $P_{(\text{云团中心},t)}$ 相对于视线的投影系数 λ。这个投影系数反映了云团相对于视线的归一化位置：

$$\lambda = \frac{\big(P(\text{云团中心}, t) - P(\text{M1}, t)\big) \cdot L_i\big(P(\text{M1}, t), (0,0,0), t\big)}{\|L_i\big(P(\text{M1}, t), (0,0,0), t\big)\|^2} \tag{22}$$

其中，$\|L_i\big(P(\text{M1}, t), (0,0,0), t\big)\|$ 是视线向量的模，$\cdot$ 是向量点积。

6.3.4 云团中心到视线投影点的距离计算

投影系数 λ 确定了云团中心在视线上的投影点 $\mathbf{P}_{\text{closest}}$：

$$\mathbf{P}_{\text{closest}} = P(\text{M1}, t) + \lambda \cdot L_i\big(P(\text{M1}, t), (0,0,0), t\big) \tag{23}$$

该点是云团中心在视线方向上的投影，它代表了与视线的最小距离点。

计算云团中心 $P(\text{云团中心}, t)$ 到投影点 $\mathbf{P}_{\text{closest}}$ 的欧几里得距离：

$$d_{\text{min}} = \big\|P(\text{云团中心}, t) - \mathbf{P}_{\text{closest}}\big\| \tag{24}$$

6.3.5 动态三维空间的遮蔽状态实时分析

6.3.5.1 单点视线遮蔽评判函数

一个目标点 T_i 在时刻 t 被遮蔽的条件是：烟幕球到导弹-目标视线的最小距离小于等于烟幕半径。定义一个二值指示函数 $\mathbb{I}_i(t)$：

$$\mathbb{I}_i(t) = \begin{cases} 1, & \text{if } d\big(\mathbf{P}(\text{云团}, t), \text{line}(\mathbf{P}(\text{采样点}N_i, t), T_i)\big) \le r, \\ 0, & \text{otherwise}, \end{cases} \tag{25}$$

其中，$\text{Obscured}_i = 1$ 表示视线被遮挡，$\text{Obscured}_i =$ 表示视线未被遮挡。

6.3.5.2 全目标整体遮蔽评判函数

1. **目标点集合**：设目标表面有 n 个采样点 $\mathbf{N}_i$，$i = 1, 2, \ldots, n$。
2. **迭代计算遮蔽状态**：对于每个目标点 $\mathbf{N}_i$，计算其是否被云团遮蔽，得到对应的遮蔽状态 Obscured_i。

$$F_{\text{full_obscuration}}(t) = \begin{cases} 1, & \text{if } \sum_{i=1}^{N} \mathbb{I}_i(t) = N, \\ 0, & \text{otherwise}. \end{cases} \tag{26}$$

6.3.5.3 动态遮蔽矩阵与遮挡标准

对于每个离散时刻 $t_n = n\Delta t$：生成一批按上述参数化得到的采样点 $\{N_i\}_{i=1}^{N}$，按照一定的遮蔽判断，计算对于某离散时刻 t_n 该目标点的遮蔽情况 $h\big(P(\text{真目标取样点}N_i), t\big) \in$

$\{0,1\}$，并且将结果存储在当前时刻 t_n 下的对应遮蔽矩阵 $M_{\text{遮蔽}}[h]_{N\times 1}$ 中。当 t_n 时刻下，在 $M_{\text{遮蔽}}[h]_{N\times 1}$ 下，$\sum_{i=1}^{N} m_{i1} = N \times 1$ 则说明该时刻下，在球形云团遮蔽下，所有采样点的视线均被遮挡，说明了真目标对象处于有效遮蔽。

6.3.6 向量化计算与并行算法

为了提高计算效率，特别是当目标点集合 $\mathcal{T}$ 很大时，可以将遮蔽状态的计算过程向量化。具体步骤包括：

- **视线向量批量计算**：将所有目标点的视线向量 $L\big(P(\text{M1},t),(0,0,0),t\big)$ 组织成一个矩阵 $\mathbf{L}(t)$，其中每一行对应一个目标点的视线向量：

$$L\big(P(\text{M1},t),(0,0,0),t\big) = \begin{bmatrix} L_1\big(P(\text{M1},t),(0,0,0),t\big) \\ L_2\big(P(\text{M1},t),(0,0,0),t\big) \\ \vdots \\ L_n\big(P(\text{M1},t),(0,0,0),t\big) \end{bmatrix} \tag{27}$$

- **投影系数批量计算**：通过矩阵运算计算所有目标点的投影系数 λ_i，从而得到批量的投影点 $\mathbf{P}_{\text{closest}}$。
- **距离计算**：批量计算每个目标点到视线的最短距离 d_{min}，并与云团的半径 R 进行比较，从而确定遮蔽状态。

在数值计算中，可以利用向量化操作和批处理计算，通过矩阵运算来实现对多点遮蔽状态的计算。这种方法能够显著提高计算效率，尤其是在目标点数量 n 较大时。

6.4 模型求解

6.4.1 时间离散化

仿真过程中，时间被离散化为一系列的时间步，通过时间步跟踪系统状态变化。

6.4.1.1 起爆时刻与仿真时长

起爆时刻定义为 t_{Boom}，表示烟幕干扰弹起爆的具体时刻，其单位为秒。仿真时长设定为 T_{end}，代表烟幕云团保持有效的最长仿真时间，单位同样为秒。

6.4.1.2 时间步长与取样点

时间被离散化为等间隔的时间点。设时间步长为 Δt，那么时间取样点 t_n 被定义为：

$$t_n = t_0 + n \cdot \Delta t, \quad n = 0,1,2,\ldots,\text{step}-1 \tag{28}$$

其中，step 为总的时间步数，$\text{step} = \frac{T_{\text{end}}}{\Delta t}$。

时间步长 $\Delta t = 0.0001$ s，对每个时间点步下的状态进行计算与跟踪，记录导弹、云团中心及目标点的状态。

6.4.2 目标采样点的空间离散化

在圆柱的侧面上，目标点的位置由极坐标参数化。假设圆柱的底面中心为 $\mathbf{C} = (C_x, C_y, C_z)$，目标点采样的数学表达为：

1. **圆周上的点**：通过角度 θ_i 均匀取样，其中 θ_i 取值范围为 $[0, 2\pi)$，且取样数为 N_θ：

$$N_{ix} = r \cdot \cos(\theta_i), \qquad N_{iy} = r \cdot \sin(\theta_i) \tag{29}$$

2. **高度上的点**：由 z_j 均匀取样，z_j 在 $[0, H]$ 范围内，样本数为 N_z：

$$N_{iz} = z_j$$

结合圆柱的中心位置 $\mathbf{C}$，每个目标点的坐标可以表示为：

$$\mathbf{N}_i = (N_{ix} + C_x,\ N_{iy} + C_y,\ N_{iz} + C_z)$$

6.4.3 动态三维离散点的实时遮蔽分析

6.4.3.1 遮蔽状态评估与分析结果

通过对三维模型中所有离散目标点和动态云团的逐时刻空间关系进行几何计算，构建一个 **二进制遮蔽矩阵** $M_{\text{遮蔽}}$。该矩阵是模型输出的核心数据结构，其维度为 $N_{\text{targets}} \times 1$，其中 N_{targets} 代表蒙特卡洛采样点的总数。矩阵中的元素 M_{i1} 的值被设定为 1 或 0，表示是否处于被遮蔽状态。

6.4.3.2 完全遮蔽持续时间量化

通过对该遮蔽矩阵进行**列向聚合分析**精确地量化烟幕在整个仿真周期内的效能。当矩阵的所有元素均为 1 时，表明在该时刻所有目标点均被烟幕完全遮蔽。通过识别并累计所有满足此条件的连续时间步，计算出总的完全遮蔽持续时间。

6.4.4 问题一最终结果

本轮仿真结果显示，**完全遮蔽持续时间为 1.405 秒**，其时间区间为 $t \in [2.943, 4.348]$ 秒。

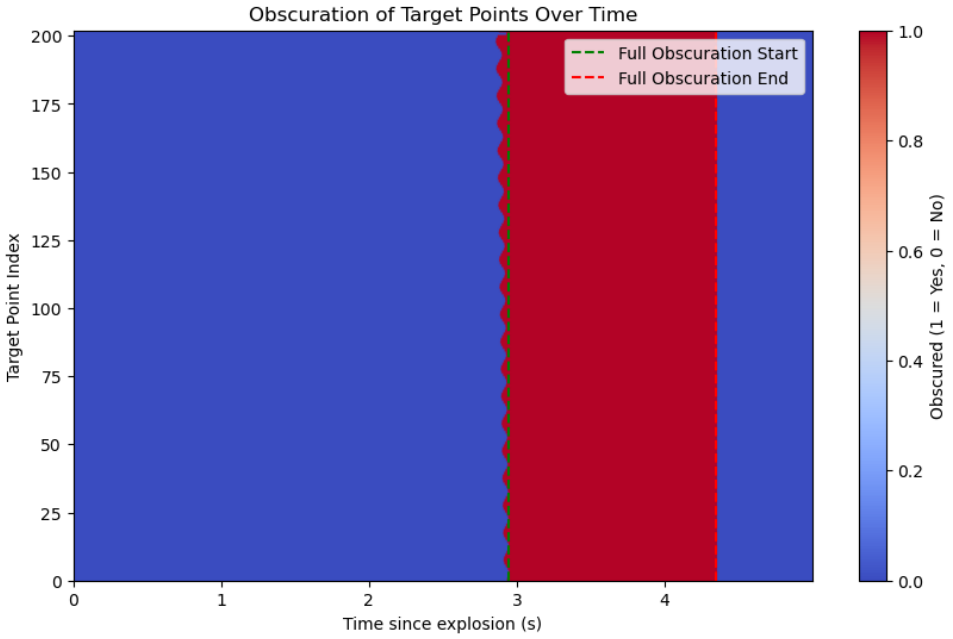


图 3　目标点遮蔽状态的时间序列分析

七、 问题二：单无人机多弹的投放策略规划

题目二的目标是通过优化决策变量 $(\theta, v_{Drop}, t_{Drop}, t_{Boom})$ 来最大化有效遮蔽时长 T_{cover}，即云团在起爆后 20 秒内 ($t_{\text{云团}} \in [0, 20]$) 覆盖真目标的总时间。约束条件有三个，分别是：无人机航向角约束 $\theta \in [0°, 360°)$，无人机速度约束 $v_{\text{Drop}} \in [70, 140]\,\text{m/s}$ 以及导弹运动时间小于导弹到达原点的时间 t_{Arrive}（t_{Arrive} 根据运动学公式进行计算）。同时，需要基于问题一建立的真目标参数化与曲面重建模型以及三维空间遮挡动态检测与视域分析模型进行建模，运用方法同章节 6.2。

7.1 投放策略的优化算法

鲸鱼优化算法（WOA）以座头鲸“包围—气泡网螺旋—随机搜索”三机制为原型，通过自适应系数在“探索—开发”间动态权衡，使个体围绕当前最优解收缩、螺旋逼近并随机跃迁以抑制早熟收敛；题目二可建模为非线性、强约束的全局优化：决策向量相互耦合决定三维运动与扩散，目标为最大化遮蔽时长 $J(x)$，具非凸、分段不光滑（离散跳变）特征，并受时序间隔、速度/航向边界与安全距离等约束；WOA 参数少、实现简、易并行，便于与高代价仿真评估耦合，在可行域内高效搜索高质量解，适配该类问题。

7.2 经典鲸鱼算法优化理论

7.2.1 包围猎物

在鲸鱼优化算法中，包围猎物这一行为模拟的核心思想是，通过当前最优个体的位置信息，引导其他个体向其靠拢，逐步形成包围圈。具体来说，每一头鲸鱼（候选解）会根据与当前最优鲸鱼的距离调整自己的游动方向与步长。

位置更新过程由以下两个公式共同描述：

$$\mathbf{D} = \|\mathbf{C} \cdot \mathbf{X}^*(t) - \mathbf{X}(t)\|$$

$$\mathbf{X}(t+1) = \mathbf{X}^*(t) - \mathbf{A} \cdot \mathbf{D}$$

其中，t 表示当前迭代次数，$\mathbf{X}^*$ 是当前最优位置向量，$\mathbf{X}$ 是当前个体位置向量。系数向量 $\mathbf{A}$ 和 $\mathbf{C}$ 由以下公式计算：

$$\mathbf{A} = 2\mathbf{a} \cdot \mathbf{r} - \mathbf{a}$$

$$\mathbf{C} = 2 \cdot \mathbf{r}$$

这里，$\boldsymbol{a}$ 是一个随迭代次数增加从 $\mathbf{2}$ 线性递减至 $\mathbf{0}$ 的控制参数，它直接影响 $\boldsymbol{A}$ 的大小和正负，从而调节搜索的广度和方向；$\boldsymbol{r}$ 则是一个在 $[\mathbf{0}, \mathbf{1}]$ 范围内随机生成的向量，用于引入随机性，避免算法过早陷入局部最优。

7.2.2 气泡网攻击

气泡网攻击是座头鲸捕食的独特策略，算法通过两种机制模拟这一行为。第一种是收缩包围机制，通过控制参数 $\mathbf{a}$ 的线性减小来实现。随着 $\mathbf{a}$ 递减，$\mathbf{A}$ 的波动范围也逐渐收缩，从而使鲸鱼的移动更加精细和局部化，有助于在最优解附近进行开发。

第二种是螺旋更新机制，模拟鲸鱼以螺旋方式向猎物游动。其位置更新公式为：

$$\mathbf{X}(t+1) = \mathbf{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \mathbf{X}^*(t)$$

其中 $\mathbf{D}' = \|\mathbf{X}^*(t) - \mathbf{X}(t)\|$ 表示当前个体与最优位置的距离，b 是定义螺旋形状的常数，l 是 $[-1, 1]$ 内的随机数。

在每一次迭代中，算法以 50% 的概率选择使用收缩包围机制或螺旋更新机制，从而平衡局部搜索的两种策略。

$$\mathbf{X}(t+1) = \begin{cases} \mathbf{X}^*(t) - \mathbf{A} \cdot \mathbf{D} & \text{if } p < 0.5 \\ \mathbf{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \mathbf{X}^*(t) & \text{if } p \geq 0.5 \end{cases}$$

7.2.3 随机搜索猎物

为了增强算法的全局探索能力，避免陷入局部最优，WOA 还引入了随机搜索机制。当系数向量 A 的绝对值大于等于 1 时，代表当前需要扩大搜索范围，此时鲸鱼不会围绕当前最优解移动，而是随机选择种群中的另一个体作为参考目标，向其方向移动。这一机制有助于跳出局部极值，探索更广阔的搜索空间。

$$\mathbf{D} = \|\mathbf{C} \cdot \mathbf{X}_{\text{rand}} - \mathbf{X}\| \tag{30}$$

$$\mathbf{X}(t+1) = \mathbf{X}_{\text{rand}} - \mathbf{A} \cdot \mathbf{D} \tag{31}$$

其中 $\mathbf{X}_{\text{rand}}$ 是当前种群中随机的一个个体位置。

WOA 通过概率 p 和系数 **A** 自适应切换三种行为，平衡全局探索与局部开发。

7.3 双阶段混合鲸鱼优化算法

本文提出的双阶段混合鲸鱼优化算法（TSH-WOA）采用“粗搜索 → 精搜索”分层框架：阶段 1 以较大时间步长与缩短仿真窗口进行快速全局摸底，锁定潜在优域；阶段 2 在精英解邻域以精细步长与完整评估窗口做局部强化，提升解精度与稳定性。机制上引入自适应高斯变异，并随迭代令前期扩大探索半径、后期微扰开发；与 WOA 的包围、螺旋、随机算子协同，实现“广搜 + 精炼”，在等精度下显著减少评估次数并提高收敛质量与鲁棒性

高斯变异公式为：$x_i' = x_i + N(0, \sigma \cdot \text{range}_i)$，该公式通过对当前解 x_i 添加一个均值为 0、标准差为 $\sigma \cdot \text{range}_i$ 的高斯随机扰动来产生新解 x_i'。这种变异策略能够在小范围内进行精细搜索，同时 σ 参数控制了变异的幅度，range_i 则确保了变异大小与参数的实际取值范围相适应。

除此之外，本优化模型还使用非线性衰减策略动态调整参数，以平衡探索与开发过程；引入随机重启机制，降低早熟收敛风险；设计自适应局部搜索概率和早停机制，提升搜索智能性并节省计算资源。

7.4 问题二最终结果

如表 1所示：

表 1　题二：无人机烟幕干扰弹投放数据

无人机运动方向	速度（m/s）	投放点坐标（m）		
		x	y	z
4.59°（x 轴为正，y 轴为正）	96.741	17928.23	10.15	1800.00

起爆点坐标（m）			有效干扰时长（s）
x	y	z	
17930.51	10.54	1800.00	4.580

八、 问题三：单无人机多弹的投放策略

问题三本质上是一个多重约束的优化问题：需要分别确认决策变量、目标函数以及物理轨迹与几何约束，综合各项来构建优化模型，从而得出最优化有效遮挡时间的烟幕干扰弹投放策略。

8.1 离散时间步的单遮蔽和联合遮蔽建模

由于第三题存在多个研究对象，分别有 3 个烟幕干扰弹 SJ_1, SJ_2, SJ_3 以及空地导弹 M_1。因此，考虑最佳投放策略，该题目需要同时考虑两种情况：

1. 情况一：单独一个烟幕干扰弹 SJ_i 可完成对于空地导弹 M_1 的有效遮挡。
2. 情况二：需要联合多个烟幕干扰弹 SJ_i，即：

$$\{SJ_a, SJ_b \mid a \neq b,\, a, b \in \{1,2,3\}\} \quad \text{以及} \quad \{SJ_1, SJ_2, SJ_3\}$$

8.1.1 单云团完全遮蔽量化建模

这里只需要任意一个烟雾球即可完成对于真目标视线的全部遮掩，并不需要借助其他烟雾球来补充视线盲区。

$$\left[F_{\text{遮}}(\mathrm{M1}, t) = 1\right] \wedge \left[\left(\mathbb{I}_{SJ_1}(t) = 1\right) \oplus \left(\mathbb{I}_{SJ_2}(t) = 1\right) \oplus \left(\mathbb{I}_{SJ_3}(t) = 1\right)\right] \tag{32}$$

8.1.2 多云团完全遮蔽量化建模

这里需要多个烟雾球进行补充，才可完成对于真目标视线的全部遮掩。

$$\left[F_{\text{遮}}(\mathrm{M1}, t) = 1\right] \wedge \overline{\left[\left(\mathbb{I}_{SJ_1}(t) = 1\right) \oplus \left(\mathbb{I}_{SJ_2}(t) = 1\right) \oplus \left(\mathbb{I}_{SJ_3}(t) = 1\right)\right]} \tag{33}$$

8.1.3 有效时间轴采样与遮蔽时长建模

在弹幕遮蔽问题中，时间上界 T_M 由初始位置差与速度的比值确定：

$$T_M = \frac{\|\mathbf{O} - P(\mathrm{M1}, t=0)\|}{300} \tag{34}$$

有效采样时间区间需满足两个约束条件：起始时刻取所有弹幕的最早出现时间，终止时刻取最晚弹幕结束时间延后 20 秒与 T_M 中的较小值。数学表达为：

$$\{t_j\}_{j=1}^{J} \subset \left[\min_k t_{e,k},\ \min\left(T_M,\ \max_k t_{e,k} + 20\right)\right) \tag{35}$$

对于单条弹幕的完全遮蔽时长，采用离散时间采样近似计算：

$$T_k \approx \sum_{j=1}^{J} \mathrm{F}_k(t_j) \cdot \Delta t \tag{36}$$

其中 $\mathrm{F}_k(t)$ 表示第 k 条弹幕在时刻 t 的遮蔽状态函数。

多条弹幕联合遮蔽时，通过联合状态函数进行积分近似：

$$T_{\cup} \approx \sum_{j=1}^{J} \mathrm{F}_{\cup}(t_j) \cdot \Delta t \tag{37}$$

此处 $\mathrm{F}_{\cup}(t)$ 表征多条弹幕在 t 时刻的共同遮蔽状态。

8.2 多变量协同优化: 决策变量、约束与可行性修复器

8.2.1 决策向量与运动学分析

8.2.1.1 决策变量定义

$$\mathbf{x} = \left[\theta, v(FY1,t), t_{\text{Drop}}^{(1)}, t_{\text{云团}}^{(1)}, t_{\text{Drop}}^{(2)}, t_{\text{云团}}^{(2)}, t_{\text{Drop}}^{(3)}, t_{\text{云团}}^{(3)}\right]^T \tag{38}$$

8.2.1.2 决策变量与运动学参数的映射关系

在干扰效果的建模过程中，需要将投放时刻、引信延迟、导弹运动以及无人机轨迹统一到连续时间轴上，从而得到云团的时空演化关系。为此，可以建立决策变量与运动学参数之间的映射。

投放与爆炸时刻之间满足延迟关系，即

$$t_{\text{Boom}}^{(n)} = t_{\text{Drop}}^{(n)} + t_{\text{云团}}^{(n)}, \quad n \in \{1,2,3\}, \tag{39}$$

其中 $t_{\text{云团}}$ 表示延迟引信在时间轴上的映射。

为了刻画目标体圆柱的空间形状，将其离散为取点集

$$\mathbb{N} = \{N_i \mid i \in [1, N]\}, \tag{40}$$

这些点可以代表圆柱外表面在计算中的近似。

导弹位置随时间向原点运动的映射关系如下：

$$\mathbf{P}(M_1, t) = \mathbf{P}(M_n, 0) + \big(- \mathbf{P}(M_n, 0)\big) v_{\mathrm{M}_n} t, \tag{41}$$

这里的 $-\mathbf{P}(M_n, 0)$ 就是导弹初始位置指向原点的方向向量。

无人机在时刻 $t_{\text{Drop}}^{(n)}$ 的投放位置为

$$P(FY_1, t_{\text{Drop}}^{(n)}) = F_0 + (\cos\theta, \sin\theta, 0) v\, t_{\text{Drop}}, \tag{42}$$

由此得到第 n 枚云团的中心在之后时刻的演化：

$$P(SJ_n, t) = \Big(x_n, y_n, z_n - s_{\text{sink}}(t - t_{\text{Drop}}^{(n)}) \Big)_+, \tag{43}$$

其中 (x_n, y_n, z_n) 即投放点位置，s_{sink} 表示云团的下沉速度，$(\cdot)_+$ 表示不取负的截断操作。

8.2.2 边界约束

无人机 FY1 的飞行角度和速度的区间界限：

s.t.

$$\begin{cases} \sum_{j=1}^{J} c_{i,j}^{V} \le S^{V} \\ \sum_{j=1}^{J} c_{m,j}^{L} \le S^{L} \\ c_{i,j}^{V} \in \{0, 1\} \\ c_{m,j}^{L} \in \{0, 1\} \end{cases} \tag{44}$$

$$\theta \in [0, 2\pi), v_d \in [70, 140] \tag{45}$$

合法取样时间轴区间界限：

$$\{t_j\}_{j=1}^{J} \subset \left[\min_k t_{e,k},\ \min\left(T_M,\ \max_k t_{e,k} + 20\right) \right] \tag{46}$$

最小投放间隔约束界限（硬约束）需要任意相邻投放间隔必须满足：

$$\Delta t \ge 1\,\text{s}, \quad t_{1,(3)} \le 60 \tag{47}$$

使用修复器 $\mathcal{R}$ 对投放时间 $(t_{Drop_1}, t_{Drop_2}, t_{Drop_3})$ 进行排序、级联推迟与整体平移/截断，使其满足链式间隔约束，相当于将可行域外的点自动投影到最近的链式间隔可行点集，提高群体搜索的可行率与稳定性。

8.2.2.1 目标函数的构建

总体的目标函数由三部分组成：主目标、形状项以及正则项，这三部分都以“秒”作为时序量纲，便于权重的解释和后续优化算法的推进。将前面得到的决策变量代入：

主目标 $\max_{n=1,2,3} T^n_{\text{Union}}(x)$ 表示让烟幕对导弹的总干扰效果，即优化器的首要任务就是最大化“有效干扰时长”，其中由容斥原理得：

$$T^n_{\text{Union}}(x) = \sum_{n=1}^{3} T^n_{\text{单体}}(x) - \left[T^{n=\{1,2\}}_{\text{重合}}(x) + T^{n=\{2,3\}}_{\text{重合}}(x) + T^{n=\{1,3\}}_{\text{重合}}(x) + T^{n=\{1,2,3\}}_{\text{重合}}(x) \right] \quad (48)$$

形状项 $\sum_{n=1}^{3} T_n(x) + \min_n T_n(x)$ 是为了鼓励每一颗弹都能发挥各自的遮蔽效果，而不是出现“存在某颗烟雾弹完全没有遮挡效果”的问题。第一个子项是三枚烟幕弹各自的单弹遮蔽时长之和，第二个子项是最大化单弹遮蔽时长的最小值，通过最大化第一个和项，优化器会避免“一个弹完成所有工作”的情况。第二个子项防止出现个别烟幕弹无效，使表现最差那一枚的遮蔽时长也要尽可能长。经过了多次网格搜索，找到了较好的权重值对 $(w_\Sigma, w_{\min})$，并整合到第二个子项得到完整的形状项如下：

$$w_\Sigma \sum_{k=1}^{3} T_k(x) + w_{\min} \min_k T_k(x) \quad \text{其中} \quad w_\Sigma = 0.22,\ w_{\min} = 0.28 \quad (49)$$

为了满足无人机投放间隔要求，定义最小投放间隔修复代价，综合考虑个体时长、投放间隔以及航向约束，为优化提供平滑且可微的指导。首先对投放时间排序：

$$\tilde{t} = \text{sort}(t_{\text{Drop1}}, t_{\text{Drop2}}, t_{\text{Drop3}}) \quad (50)$$

然后按最小间隔 $\delta = 1\,\text{s}$ 逐个修复：

$$R_1 = \tilde{t}_1, \quad R_2 = \max(\tilde{t}_2, R_1 + \delta), \quad R_3 = \max(\tilde{t}_3, R_2 + \delta) \quad (51)$$

并裁剪至投放窗 $[0, T_F]$，最终代价为：

$$\Pi_2(x) = 0.05 \left\| \text{sort}(t_{\text{Drop1}}, t_{\text{Drop2}}, t_{\text{Drop3}}) - R(t_{\text{Drop1}}, t_{\text{Drop2}}, t_{\text{Drop3}}) \right\|_1 \quad (52)$$

用于惩罚投放间隔不足的情况。

为了引导优化算法沿合理方向搜索，设计航向正则项：

$$\mu = \text{atan2}\left((T_c - F_0)_y, (T_c - F_0)_x\right), \quad \Pi_3(x) = 0.02\ \text{wrap}(\theta - \mu)^2 \quad (53)$$

其中 $\text{wrap} : \mathbb{R} \to [-\pi, \pi)$，使角度偏差限制在 $[-\pi, \pi)$ 范围内，从而抑制无谓游走。

三个正则项合并得到总正则：

$$\Pi(x) = \Pi_1(x) + \Pi_2(x) + \Pi_3(x) \quad (54)$$

综合前面的优化建模过程，得到了优化方程：

$$\max J(x) = T_{\cup}(x) + w_{\Sigma}\sum_{k=1}^{3} T_k(x) + w_{\min}\min_k T_k(x) - \Pi(x),$$

s.t.

$$\begin{cases} \theta \in [0, 2\pi), \\ v_d \in [70, 140], \\ \{t_j\}_{j=1}^{J} \subset [\min_k t_{e,k},\ \min(T_M,\ \max_k t_{e,k} + 20))\,, \\ t_{\text{Drop2}} \geq t_{\text{Drop1}} + 1, \quad t_{\text{Drop3}} \geq t_{\text{Drop2}} + 1, \quad t_{\text{Drop3}} \leq 60 \end{cases}$$

8.2.2.2 元启发式算法

基于上述设定，数值评估时采用固定时间网格 $\Delta t = 0.01, 0.02$ s 对目标点进行采样收集。在遮蔽判定环节中，用 softmin 软化近似替代硬阈值与硬逻辑，使得在不可导或非光滑的位置，黑箱优化依然能够稳定地比较候选解的优劣。

考虑到投放时刻必须满足最小间隔 $\delta = 1\,\text{s}$，将区间 $[0, T_F]$ 进行重参数化，把间隔约束直接融入参数表达：

$$\tilde{t}_{\text{Drop1}} = \text{sigmoid}(u_1)T_F, \quad \tilde{t}_{\text{Drop2}} = \tilde{t}_{\text{Drop1}} + \delta + \text{softplus}(u_2), \quad \tilde{t}_{\text{Drop3}} = \tilde{t}_{\text{Drop2}} + \delta + \text{softplus}(u_3) \tag{55}$$

之后再裁剪至 $[0, T_F]$，使得搜索空间天然满足间隔约束，且 Π_2 可以作为稳定因子。

在优化流程上，首先利用优化算法进行全局探索，通过 50–150 次评估即可较快锁定无人机的航向 θ、速度 v 以及 $\{t_{\text{Drop1}}, t_{\text{Drop2}}, t_{\text{Drop3}}, t_b^{(1:3)}\}$ 的大致优区，其中多次启动点围绕先验航向

$$\mu = \text{atan2}\left((T_c - F_0)_y, (T_c - F_0)_x\right),$$

展开，速度则在平台可行上下界的中值附近选取。随后进入分块微调阶段，交替优化 $\{\theta, v\}$ 与 $\{t_{\text{Drop}k}, t_b^{(k)}\}$，通过坐标下降或信赖域邻域搜索快速提升 $T_{\cup}$，同时保持三枚干扰弹各自遮蔽时长 T_k 的平衡。最后，在 9–12 维空间中采用 Nelder–Mead 算法收敛，并结合 Deb 的“可行优先”规则筛选候选解：先比较个体的时长下限是否满足要求，再对整体目标进行比较。

通过这种“固定网格 + 软化逻辑 + 重参数化 + 三阶段搜索”的组合方法，可以在题目所给的无人机 FY1 投放三枚烟幕弹、对目标进行视线遮挡的场景下，实现联合完全遮蔽时长的稳定最大化。进一步结合运动轨迹方程的转换，就能够得出第三问所需的投放策略。

8.3 问题三模型结果

经过运算，最大有效遮蔽时间为 6.74s，具体投放策略如下表 2

表 2　题三：单无人机多干扰弹投放策略

无人机运动方向	无人机速度（m/s）	投放点坐标（m）		
		x	y	z
3.842	74.394	17800.00	0.00	1800.00
3.842	74.394	17874.23	4.97	1800.00
3.842	74.394	17948.45	9.95	1800.00

起爆点坐标（m）			有效干扰时长（s）
x	y	z	
17930.76	0.78	1800.00	4.58
17887.34	5.85	1800.00	4.24
17948.45	9.95	1800.00	4.02

九、 问题四：多无人机单干扰弹的投放策略优化

9.1 基于题目三更新优化方程

问题四需要基于问题三更新决策变量与运动学参数的映射关系、烟雾弹初始位置和边界约束

9.1.1 更新决策变量

在本体中，决策变量变为对于三台不同无人机的参数设定，从而来决定烟雾球运动轨迹，进一步影响到烟雾球的遮挡效果。

对每架无人机 $k \in \{1,2,3\}$（分别为 FY1、FY2、FY3）：

- 航向角 $\theta_k \in (-\pi, \pi]$（接到任务后瞬时设定，随后不再改变）
- 速度 $v_k \in [70, 140]$ m/s（等高度匀速直线飞行）
- 投放时刻 $t_{\text{Drop},k} \geq 0$
- 延时起爆 $t_{b,k} \geq 0$，起爆时刻 $t_{e,k} = t_{\text{drop},k} + t_{b,k}$

因此无人机优化投放的决策变量为：

$$\text{对于} n \in \{1,2,3\}，\text{有} \mathbf{x}_n = [\theta_n, v_n, t_{\text{drop},n}, t_{b,n}] \tag{56}$$

由于 3 颗烟雾弹由不同的无人机挂载，而不同的无人机具有不同的三维空间初始位置，所以 3 颗烟雾弹的初始位置与三台不同无人机初始位置相同。

9.1.2 更新边界约束

取消最小投放 1s 间隔修复代价：由于不同无人机进行投放，且每一台无人机 FY1，FY2，FY3 分别各自只投一颗烟雾弹，所以并不需要考虑投放间隔的题目限制，因此全新的正则与惩罚项只有两个子项：

$$\Pi'(x) = \Pi_1(x) + \Pi_3'(x) \tag{57}$$

其中：

- 个体下限软约束：$\Pi_1 = \sum_k [T_{\min} - T_k]_+$，$T_{\min} = 0.9\text{s}$（确保三机均有贡献）。
- 航向先验正则：$\Pi_3 = \text{wrap}(\theta_k - \mu_k)^2$，$\mu_k = \text{atan2}((T_c - F_{k,0})_y, (T_c - F_{k,0})_x)$。

9.1.3 更新优化方程

$$\max J'(x) = T_{\cup}(x) + w_{\Sigma} \sum_{k=1}^{3} T_k(x) + w_{\min} \min_k T_k(x) - \Pi(x), \tag{58}$$

s.t.

$$\begin{cases} \theta \in [0, 2\pi), \\ v_d \in [70, 140], \\ \{t_j\}_{j=1}^{J} \subset [\min_k t_{e,k}, \min(T_M, \max_k t_{e,k} + 20)) \end{cases} \tag{59}$$

9.2 基于骨架反演与贪心组合的混合优化方法

本题采用一种改进的分层混合优化策略，将原本连续、高维黑箱优化问题，巧妙地拆解为一系列可管理的子问题，并为每个阶段设计了特定的优化算法。

9.2.1 候选参数化与“走廊”反解：

为把搜索空间集中在“导弹 → 目标”的走廊，选择先在 X-Y 平面设定“起爆时刻 t_{boom} 的走廊点”，随后由几何反解无人机方向与速度：

$$\theta = \text{atan2}\Big(\big(P_{xy}(\text{SJ}_n, t) - P_{xy}(\text{假目标}, t)\big)_y,\ \big(P_{xy}(\text{SJ}_n, t) - P_{xy}(\text{假目标}, t)\big)_x \Big) \square$$

$$v = \frac{\Big\| p_{xy}(\text{SJ}_n, t) - p_{xy}(\text{假目标}, t) \Big\|}{t_{\text{云团},n}}, \quad t_{\text{drop},n} = t_{\text{云团},n} - t_{\text{boom},n} \tag{60}$$

若上面搜索空间取样的 v 值超界，回调 $t_{\square\square n}$，使 v 入界。这骨架反搜索是后续候选构造的核心。

9.2.2 单机候选池构造

单机候选池构造包含粗筛，中评，兜底，加密四个阶段。估计与导弹位置 $P(\mathrm{FY}_n, t)$ x 值对齐的 $t_{\text{云团},n}$，并构造局部网格，枚举可行解的向量。对 $\left(t_{\text{云团},n},\, \beta,\, t_{\text{boom},n}\right)$，采用了三个阶段的分阶段优化可行解：

粗筛阶段： 运用“锥近似”建模快速评估计算，保留下单弹总时长 $\geq 0.2\,\mathrm{s}$ 的组合。

中评阶段： 运用“视线–烟幕球”的遮蔽量化评估，保留 $T \geq 0.5\,\mathrm{s}$ 且有连续区间者。

兜底阶段： 若为空，放宽范围与分辨率再来一次。本题遇到了无人机 FY2 投放的烟幕干扰弹，在前期的搜索空间中，并没有任何候选的可行解组合，所以需要再扩大采样。

$$C_k:\ \text{带时间区间的} k \text{ 个高质量候选时间段} \Big\{ \left(C_{i,\min},\, C_{i,\max}\right) \mid i \in [1,k] \Big\}.$$

时段加密： 围绕 Top-K 的 $t_{\text{云团},n}$ 做局部细化并去重，这样就得到 k 个“带时间区间”的高质量候选时间区间段集合：

$$C_k.$$

9.2.3 候选空间中组合搜索

在本题“三机单弹协同遮蔽”情境下，先把每个候选方案（由 FY1/FY2/FY3 的起爆时刻区间映射成时间掩码）视为一段覆盖片段，随后在离散层面对“并集掩码长度 L”做组合优化。按步骤进行组合优化：

1. **贪心搜索**：每轮挑选能带来最大的“新增覆盖”候选（每轮至多取一枚），快速获得强基线；
2. **Beam 法**：先在某一轮取 Top-B1，再与下一轮逐一合并并且择优筛成 Top-B2，最后拼第三轮，既保留前瞻性又避免三元组的爆炸枚举；
3. **对偶 → 拼第三（pair-then-third）**：先按“双指标（单弹完全遮蔽时长 + 掩码长度）”为每轮取 Top-K，枚举两轮所有对子并计算其并集长度，截取 Top-P 的最优对子，再逐个对子与第三轮候选尝试拼接以逼近“全三元组”最优解而显著降复杂度；
4. **离散 1/2-opt**：在候选池内做局部重定形：1-opt 固定其他两轮、单轮重选以增大并集；2-opt 固定第三轮、两轮对内穷举组合以增大并集，交替迭代至收敛。

9.2.4 问题四建模结果

上述离散阶段仅用 L 作为快速代理目标以高效筛选与整形覆盖结构，最终再用严格几何评估计算联合完全遮蔽时长 $T_U(x)$ 做复核与取舍，必要时用连续微调替换离散解，

从而在三机协同下获得既高覆盖又物理一致的最优方案，最大有效遮蔽时间为 8.760s，得到最佳的无人机投放方案如表 3

表 3　题四：多无人机多干扰弹的优化策略

无人机编号	无人机运动方向 (rad)	无人机运动速度 (m/s)	投放点 x 坐标	投放点 y 坐标	投放点 z 坐标
FY1	3.122828	124.839	17616.28	3.45	1800.00
FY2	5.061496	93.41	12376.69	365.20	1400.00
FY3	1.661075	78.091	5757.31	-319.07	700.00

无人机编号	起爆点 x 坐标	起爆点 y 坐标	起爆点 z 坐标	有效干扰时长 (s)
FY1	17014.04	14.75	1709.20	2.930
FY2	12505.98	10.00	1256.97	3.680
FY3	5725.75	29.50	577.38	2.155

十、 问题五：多导弹-多无人机—多干扰弹的投放策略

导弹集合 $\mathcal{I} = \{\mathrm{M1}, \mathrm{M2}, \mathrm{M3}\}$、无人机集合 $\mathcal{K} = \{\mathrm{FY1}, \mathrm{FY2}, \mathrm{FY3}, \mathrm{FY4}, \mathrm{FY5}\}$。每架无人机最多可投放 $B_k = 3$ 枚烟幕弹。对每枚导弹 $\mathrm{M}_i \in \mathcal{I}$ 预先生成候选起爆事件集合 $\mathcal{E}_{\mathrm{M}_i}$。每个事件 $e \in \mathcal{E}_{\mathrm{M}_i}$ 含参数

$$t_e^{\mathrm{exp}}, \quad E_e^{xy} \in \mathbb{R}^2, \quad E_e^z \in \mathbb{R}, \quad \mathcal{I}_e \subset \mathbb{R}_{\geq 0}, \quad s_e = \mathrm{len}(\mathcal{I}_e) \in \mathbb{R}_{\geq 0}. \tag{61}$$

导弹 M_i 到达假目标时间 $t_{end}(M_i)$ 事件 e 的云团球心满足 $\mathrm{P}(\text{cloud } e, t_e^{\mathrm{exp}}) = \left(E_e^{xy}, E_e^z\right)$,

$$[\mathrm{P}(\text{cloud } e, t)]_{xy} \equiv E_e^{xy}, \quad [\mathrm{P}(\text{cloud } e, t)]_z = E_e^z - v_{\mathrm{sink}} \, (t - t_e^{\mathrm{exp}})_+$$

遮蔽判据为若不存在目标圆柱体表面点 N 使 $\overline{\mathrm{P}(\mathrm{M}_i, t)\, N} \cap \mathbb{B}(\mathrm{P}(\text{cloud } e, t),\, R) = \varnothing$ 则 $t \in \mathcal{I}_e$，并令 $s_e = \mathrm{len}(\mathcal{I}_e)$。对给定无人机—事件对 (FY_k, e) 定义可达指示量 $\mathrm{edge}_{\mathrm{FY}_k, e} \in \{0, 1\}$：当且仅当存在 $v_k \in [v_{\min}, v_{\max}]$、单位向量 $\mathrm{d}(\mathrm{FY}_k, 0)$、引信延时 $\tau_e \geq 0$ 使

$$[\mathrm{P}(\mathrm{FY}_k, t_e^{\mathrm{exp}})]_{xy} = E_e^{xy}, \quad E_e^z = z_k - \tfrac{1}{2} g \, \tau_e^2, \quad t_e^{\mathrm{drop}} = t_e^{\mathrm{exp}} - \tau_e \geq 0 \tag{62}$$

则 $\mathrm{edge}_{\mathrm{FY}_k, e} = 1$，否则为 0。同一无人机两事件 $e \neq e'$ 需满足 $|t_e^{\mathrm{drop}} - t_{e'}^{\mathrm{drop}}| \geq \Delta$。

10.1 无人机任务目标分配

定义无人机对导弹的适配度如下：

$$a_{k,i} \triangleq \max_{e \in \mathcal{E}_{\mathrm{M}_i}} \; s_e \, \mathrm{edge}_{\mathrm{FY}_k, e} \tag{63}$$

若 $\max_{e \in \mathcal{E}_{\mathrm{M}_i}} \mathrm{edge}_{\mathrm{FY}_k, e} = 0$ 则认为 $a_{k,i} = 0$。令 $y_{k,i} \in \{0,1\}$ 表示无人机 FY_k 在战略层被指派支援导弹 M_i，并令 $q_i \in \mathbb{Z}_{\geq 0}$ 为导弹 M_i 的无人机配额，满足 $\sum_{i \in \mathcal{I}} q_i = |\mathcal{K}|$。战略层优化为

$$\max_{y,q} \sum_{k \in \mathcal{K}} \sum_{i \in \mathcal{I}} a_{k,i} \, y_{k,i} \quad \text{s.t.} \sum_{i \in \mathcal{I}} y_{k,i} = 1, \; \sum_{k \in \mathcal{K}} y_{k,i} = q_i, \; q_i \geq 1, \; y_{k,i} \in \{0,1\} \tag{64}$$

10.2 资源调度与协同规划

10.2.1 目标分配的决策延伸

对每个导弹 M_i 记其被选事件的时间区间并集为

$$\mathcal{S}_i \triangleq \bigcup_{k \in \mathcal{K}} \bigcup_{\{e \in \mathcal{E}_{\mathrm{M}_i} \, : \, x_{k,e} = 1\}} \mathcal{I}_e, \quad T_i \triangleq \mathrm{len}(\mathcal{S}_i) \tag{65}$$

其中 $x_{k,e} \in \{0,1\}$ 为决策层的事件选取变量。采用词典序目标

$$\max \; \big(T_{\min}, \; \sum_{i \in \mathcal{I}} T_i \big) \tag{66}$$

其中 $\quad T_{\min} \leq T_i, \; \forall i \in \mathcal{I}$。

10.2.2 资源调度的可行域

以“航向—等速一致性”对每架无人机的可行事件作预聚类 $\{\mathcal{C}_{k,m}\}_m$，可行约束为

$$\sum_{e} x_{k,e} \leq B_k, \quad \forall \, e \neq e' \tag{67}$$

且

$$|t_e^{\mathrm{drop}} - t_{e'}^{\mathrm{drop}}| < \Delta : \; x_{k,e} + x_{k,e'} \leq 1, \tag{68}$$

$z_{k,m}$ 表示无人机 k 是否选择第 m 个“直线—等速”航迹簇；只能选且必须选一条：

$$\sum_{m} z_{k,m} = 1, \qquad \forall \, k \in \mathcal{K}.$$

若未选择某簇（$z_{k,m} = 0$），该簇里的事件都不能被选：

$$x_{k,e} \leq z_{k,m}, \qquad \forall \, k \in \mathcal{K}, \; \forall \, m, \; \forall \, e \in \mathcal{C}_{k,m}.$$

事件还必须可达（仅当 $\text{edge}_{\text{FY}_k,e}=1$ 才能选）：

$$x_{k,e} \le \text{edge}_{\text{FY}_k,e}, \qquad \forall k \in \mathcal{K},\ \forall e.$$

变量取值为 0–1：

$$x_{k,e},\ z_{k,m} \in \{0,1\}, \qquad \forall k \in \mathcal{K},\ \forall m,\ \forall e.$$

10.2.3 协同规划的增量准则

为避免多机在同一时间窗重复遮蔽，定义对导弹 M_i 的边际增益

$$\Delta_i(e \mid \mathcal{S}_i) \triangleq \text{len}\big(\mathcal{S}_i \cup \mathcal{I}_e\big) - \text{len}(\mathcal{S}_i) \tag{69}$$

在满足可行域与战略层指派 $y_{k,i}$ 的前提下，按“两阶段—全局协同”顺序选取事件：先在各自主责导弹上，贪心选择能最大化 $\Delta_i(e \mid \mathcal{S}_i)$ 的 (FY_k, e)；再在全局范围内，对剩余弹槽继续选择能最大化 $\max_{i\in\mathcal{I}} \Delta_i(e \mid \mathcal{S}_i)$ 的 (FY_k, e)。若某无人机的边际增益均为零或被一致性/间隔约束阻塞，则不强行用满 B_k。

10.3 问题五建模结果

根据上述分析，最大有效遮蔽时间为 19.8523s，得到最佳的无人机投放方案如表 4。

十一、 模型的评价

11.1 模型的优点

- 参数化目标表示：圆柱体采样可均匀覆盖目标表面，提高遮蔽判定的准确性与可解释性。
- 扩展性与通用性强：通用运动轨迹、动态矩阵和优化算法不仅适用于单弹单机场景，也可直接扩展到多弹多机协同投放
- 分阶段优化易实现：先全局搜索粗策略，再局部精调，可降低复杂度并提升收敛速度。

11.2 模型的缺点

- 理想化假设多：忽略了风速、烟幕扩散非均匀性和导弹制导误差，可能与实际情况存在偏差。
- 精度—成本权衡：高精度时间步长和碰撞检测计算量大，过粗又可能漏判短时遮蔽窗口。

表 4　题五：无人机烟幕干扰弹投放数据

无人机航迹参数

无人机编号	无人机运动方向 (°)	无人机运动速度 (m/s)
FY1	179.9568	119.9308
FY2	182.1471	115.3494
FY3	179.1950	110.1914
FY4	181.2171	130.2323
FY5	178.0821	124.5956

烟幕弹投放/起爆与效果（续表）

无人机编号	烟幕弹编号	投放点 x(m)	投放点 y(m)	投放点 z(m)	起爆点 x(m)	起爆点 y(m)	起爆点 z(m)	有效干扰时长 (s)	干扰导弹编号
FY1	1	17208.1521	10.7414	1799.9821	17027.1819	10.4826	1788.9263	11.9566	M1
FY1	2	16488.3111	10.4126	1800.1629	16308.6896	10.3274	1788.8604	9.7971	M1
FY1	3	15768.4024	11.1678	1800.0058	15588.1556	10.6222	1788.7603	7.9375	M2
FY2	1	17110.2519	-73.8170	1500.0347	16938.5730	-80.8801	1489.0433	11.6971	M1
FY2	2	16419.0067	-98.5861	1499.8718	16249.0589	-103.7165	1488.7931	10.1695	M2
FY2	3	15730.5594	-122.3023	1499.9589	15558.9462	-127.6891	1489.0531	8.0700	M2
FY3	1	16839.6616	91.0164	699.9342	16675.4752	94.2232	689.2908	11.8361	M2
FY3	2	16179.4318	103.5751	700.1991	16015.5802	106.5019	689.1634	9.9812	M3
FY3	3	15519.3055	114.1833	700.1334	15354.3161	117.4589	688.9992	7.9369	M3
FY4	1	17321.7438	106.8211	1799.9147	17124.7552	102.4018	1789.0971	11.8866	M2
FY4	2	16540.1884	93.2988	1799.8987	16345.0617	88.0821	1788.8122	9.8865	M1
FY4	3	15760.6474	79.9968	1799.9974	15565.5690	75.8757	1789.1156	8.1787	M3
FY5	1	16851.0892	-63.5586	1200.1718	16663.0502	-55.9146	1188.9135	12.3186	M3
FY5	2	16100.9943	-38.0121	1199.8420	15913.4223	-30.1108	1189.0780	10.1378	M3
FY5	3	15349.7075	-10.9785	1200.0778	15163.5999	-5.3716	1188.9634	8.3450	M1

- 模型复杂度高：参数较多（航向、速度、投放时间、起爆时间），导致调试和验证较困难。

参考文献

[1] ChatGPT, ChatGPT5, OpenAI, 2025-09-06[EB/OL].

[2] LAINE S. A general algorithm for output-sensitive visibility preprocessing[C/OL]// LASTRA A, OLANO M, LUEBKE D P, et al. I3D ’05: Proceedings of the 2005 Symposium on Interactive 3D Graphics and Games. Association for Computing Machinery, 2005: 31-40. DOI: 10.1145/1053427.1053433.

[3] LEWIS R M, TORCZON V. Icase report no. 96-20 pattern search algorithms for bound constrained minimization: 198306[R/OL]. National Aeronautics and Space Adminis-

tration, Langley Research Center, 1996. https://ntrs.nasa.gov/api/citations/19960022270/downloads/19960022270.pdf.

[4] LEWIS R M, TORCZON V. Icase report no. 96-20 pattern search algorithms for bound constrained minimization: 198306[R/OL]. National Aeronautics and Space Administration, Langley Research Center, 1996. https://ntrs.nasa.gov/api/citations/19960022270/downloads/19960022270.pdf.

[5] O ' ROURKE J. Visibility[M/OL]//GOODMAN J E, O ' ROURKE J, TóTH C D. Handbook of Discrete and Computational Geometry. 3rd ed. CRC Press, 2017: 875-896. https://www.csun.edu/~ctoth/Handbook/chap33.pdf.

附录 A　文件列表

文件名	功能描述
result1.xlsx	问题三投放策略
result2.xlsx	问题四投放策略
result3.xlsx	问题五投放策略

附录 B　代码

Q1.py

```python
import numpy as np
import matplotlib.pyplot as plt

# --------------------
# 参数初始化
# --------------------
M0 = np.array([20000.0, 0.0, 2000.0])   # 导弹初始位置 (x, y, z)
F0 = np.array([17800.0, 0.0, 1800.0])   # 无人机初始位置 (x, y, z)
v_M = 300.0     # 导弹速度 (m/s)
v_F = 120.0     # 无人机速度 (m/s)

# 目标参数：圆柱体（半径7m，高度10m），底面中心坐标 (0, 200, 0)
target_center = np.array([0.0, 200.0, 0.0])
target_radius = 7.0     # 真目标半径 (m)
target_height = 10.0    # 真目标高度 (m)

# 烟幕参数
R = 10.0          # 烟幕球有效半径 (m)
sink_speed = 3.0    # 烟幕下沉速度 (m/s)
drop_time = 1.5     # 无人机投弹延时 (s)
expl_delay = 3.6    # 烟幕弹引信延时 (s)
expl_time = drop_time + expl_delay    # 起爆时刻 (s)
```

```
23
24  # --------------------
25  # 计算无人机的投弹点和起爆点坐标
26  # --------------------
27  # 计算无人机水平飞行方向单位向量（忽略高度差）
28  direction_F = np.array([0.0, 0.0, 0.0]) - F0
29  direction_F[2] = 0.0  # 水平分量
30
31  # 方向向量
32  direction_F = direction_F / np.linalg.norm(direction_F)
33
34  # 投弹点：无人机从F0出发，沿水平方向飞行drop_time秒的位置
35  P_drop = F0 + direction_F * v_F * drop_time
36  # 烟幕弹起爆点：从投弹点出发，经过expl_delay秒的水平位移，加上
       下坠高度
37  horizontal_vel = direction_F * v_F
38  P_expl = P_drop + horizontal_vel * expl_delay
39  vertical_drop = 0.5 * 9.81 * (expl_delay ** 2)  # 下坠距离 =
       1/2 * g * t^2
40  P_expl[2] = P_drop[2] - vertical_drop
41
42  # --------------------
43  # 定义导弹和烟幕运动模型
44  # --------------------
45  # 导弹飞行方向单位向量（从导弹初始位置指向原点(0,0,0)）
46  direction_M = np.array([0.0, 0.0, 0.0]) - M0
47  direction_M = direction_M / np.linalg.norm(direction_M)
48  # 导弹到达原点的总飞行时间
49  total_flight_time = np.linalg.norm(M0 - np.array([0.0, 0.0,
       0.0])) / v_M
50
51  def missile_position(time):
52      """
53      给定绝对时间time，返回导弹在该时刻的位置坐标。
54      如果导弹已到达目标（原点），则保持在原点位置。
```

```
    """
    if time >= total_flight_time:
        return np.array([0.0, 0.0, 0.0])
    return M0 + direction_M * v_M * time

def smoke_center(time_after_explosion):
    """
    给定起爆后经过的时间，返回此刻烟幕球中心的位置。
    起爆前返回None；假设烟幕在起爆后20秒内保持有效。
    """
    if time_after_explosion < 0 or time_after_explosion > 20:
        return None
    # 烟幕中心水平位置固定为起爆点P_expl的水平坐标，高度随时间线性下降
    return np.array([P_expl[0], P_expl[1], P_expl[2] - sink_speed * time_after_explosion])

# --------------------
# 构建目标表面的采样点集合
# --------------------
def sample_cylinder_surface(radius, height, num_points_per_circle=10, num_height_samples=10):
    """
    在圆柱侧面以及顶/底面中心取样点。
    - num_points_per_circle: 每个高度截面上均匀取样的点数（分布在圆周上）
    - num_height_samples: 沿高度方向取样的层数
    返回numpy数组，形状为(N, 3)
    """
    points = []
    # 圆周角度和高度取样
    angles = np.linspace(0, 2 * np.pi, num_points_per_circle, endpoint=False)
    heights = np.linspace(0, height, num_height_samples)
    # 圆柱侧面采样
```

```
    for z in heights:
        for angle in angles:
            x = radius * np.cos(angle)
            y = target_center[1] + radius * np.sin(angle)
            points.append([x, y, z])
    # 顶面和底面中心点
    points.append([target_center[0], target_center[1], 0.0])  # 底面中心
    points.append([target_center[0], target_center[1], height])  # 顶面中心
    return np.array(points)

# 生成目标采样点（例如圆周上10点，垂直方向10层，共100个侧面点，加上顶/底中心点）
target_points = sample_cylinder_surface(target_radius, target_height, num_points_per_circle=10, num_height_samples=20)

# --------------------
# 仿真计算：扫描起爆后每个时刻的遮蔽状态
# --------------------
# 设置仿真时间参数
start_time = 0.0           # 起爆时刻作为时间0
end_time = 5.0             # 起爆后模拟5秒（可根据需要调整）
dt = 0.0001                # 时间步长（秒）

times = np.arange(start_time, end_time, dt)
num_times = len(times)
num_targets = len(target_points)

# 用于记录每个目标点在每个时刻是否被遮蔽的矩阵 (num_targets x num_times)
obscured_matrix = np.zeros((num_targets, num_times), dtype=int)

```

```
# 遍历每个时间步，计算目标各点是否被烟幕遮挡
for idx, t in enumerate(times):
    # 当前绝对时间 = 起爆时刻expl_time + t
    current_time = expl_time + t
    missile_pos = missile_position(current_time)
    smoke_pos = smoke_center(t)  # t是起爆后经过的时间

    if smoke_pos is None:
        # 烟幕未生成或已无效，则所有目标点均未被遮蔽Z
        obscured_matrix[:, idx] = 0
    else:
        # 向量化计算每个目标点到导弹视线与烟幕球的距离
        # 目标点相对导弹的位置向量
        rel_vectors = target_points - missile_pos   # 形状 (num_targets, 3)
        rel_dir_norm2 = np.einsum('ij,ij->i', rel_vectors, rel_vectors)  # 每个向量的平方长度
        # (烟幕中心 - 导弹) 相对导弹的向量
        c_minus_m = smoke_pos - missile_pos
        # 计算每个目标方向与烟幕中心方向的投影参数lambda_opt
        # lambda_opt = (C - M)·(T - M) / ||T - M||^2
        lambda_opt = (rel_vectors @ c_minus_m) / rel_dir_norm2
        # 将lambda限制在[0,1]区间（线段范围内）
        lambda_opt = np.clip(lambda_opt, 0.0, 1.0)
        # 计算视线(导弹M到目标点T)上距烟幕中心C最近的点
        closest_points = missile_pos + rel_vectors * lambda_opt[:, None]  # 形状(num_targets,3)
        # 计算烟幕中心到该最近点的距离
        dist2 = np.einsum('ij,ij->i', smoke_pos - closest_points, smoke_pos - closest_points)
        distances = np.sqrt(dist2)
        # 判断每个目标点是否被遮蔽（距离<=烟幕球半径R表示视线被烟幕阻挡）
        obscured_matrix[:, idx] = (distances <= R).astype(int)

```

```
# --------------------
# 分析结果：计算目标完全遮蔽的时间区间及持续时间
# --------------------
# 找出每个时刻是否所有目标点都被遮蔽（行向量全为1）
full_cover_flags = np.all(obscured_matrix == 1, axis=0)  # 布尔数组，长度=num_times
covered_indices = np.where(full_cover_flags)[0]

full_obscuration_intervals = []
total_full_obsc_duration = 0.0

if covered_indices.size > 0:
    # 寻找连续的遮蔽区间
    start_idx = covered_indices[0]
    for i in range(1, len(covered_indices)):
        if covered_indices[i] != covered_indices[i-1] + 1:
            # 遮蔽连续段在前一个索引结束
            end_idx = covered_indices[i-1]
            # 记录区间的实际时间（相对起爆时刻）
            interval_start = times[start_idx]
            interval_end = times[end_idx]
            full_obscuration_intervals.append((interval_start, interval_end))
            total_full_obsc_duration += (interval_end - interval_start)
            # 新的连续段开始
            start_idx = covered_indices[i]
    # 处理最后一个连续区间
    end_idx = covered_indices[-1]
    interval_start = times[start_idx]
    interval_end = times[end_idx]
    full_obscuration_intervals.append((interval_start, interval_end))
    total_full_obsc_duration += (interval_end - interval_start)
```

```
173
174 # 打印完全遮蔽的时间区间和总持续时间
175 if full_obscuration_intervals:
176     print("Full obscuration intervals (s):",
177           [(f"{interval[0]:.3f}", f"{interval[1]:.3f}") for
    interval in full_obscuration_intervals])
178     print(f"Total full obscuration duration: {
    total_full_obsc_duration:.3f} seconds")
179 else:
180     print("No time interval found where all target points are
    fully obscured.")
181
182 # --------------------
183 # 可视化：绘制目标各采样点的遮蔽状态热力图
184 # --------------------
185 plt.figure(figsize=(10, 6))
186 # 使用imshow展示遮蔽矩阵，x轴为时间，y轴为目标采样点索引
187 # 为了将x轴刻度转换为实际秒数，这里设置extent参数
188 plt.imshow(obscured_matrix, aspect='auto', cmap='coolwarm',
    origin='lower',
189            extent=[times[0], times[-1], 0, num_targets])
190 plt.colorbar(label="Obscured (1 = Yes, 0 = No)")
191 plt.xlabel('Time since explosion (s)')
192 plt.ylabel('Target Point Index')
193 plt.title('Obscuration of Target Points Over Time')
194
195 # 在图上标注完全遮蔽区间的起止时刻
196 if full_obscuration_intervals:
197     for (t_start, t_end) in full_obscuration_intervals:
198         plt.axvline(x=t_start, color='green', linestyle='--',
    label='Full Obscuration Start')
199         plt.axvline(x=t_end, color='red', linestyle='--',
    label='Full Obscuration End')
200     # 添加图例（避免重复标签）
201     handles, labels = plt.gca().get_legend_handles_labels()
```

```
    # 只保留唯一标签
    by_label = dict(zip(labels, handles))
    plt.legend(by_label.values(), by_label.keys(), loc='upper right')

plt.show()
```

Q2.py

```
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from tqdm import tqdm
import time
from typing import Tuple
from mpl_toolkits.mplot3d import Axes3D
from concurrent.futures import ProcessPoolExecutor
import multiprocessing
from functools import lru_cache
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# ==========================
# 常量 & 场景参数
# ==========================
g = 9.81

# M1 与 FY1 初始状态
M0 = np.array([20000.0, 0.0, 2000.0], dtype=np.float32)
F0 = np.array([17800.0, 0.0, 1800.0], dtype=np.float32)

# —— 真目标（圆柱）——
target_center = np.array([0.0, 200.0, 0.0], dtype=np.float32)
target_radius = 7.0
target_height = 10.0

```

```python
# —— 云团参数——
R = 10.0
sink_speed = 3.0
valid_after_explosion = 20.0

# M1 速度向量（固定）
sqrt101 = np.sqrt(101.0)
v_M_vec = 300.0 * np.array([-10.0/sqrt101, 0.0, -1.0/sqrt101],
    dtype=np.float32)
T_M_total = np.linalg.norm(M0 - np.array([0.0, 0.0, 0.0],
   dtype=np.float32)) / 300.0

# ==========================
# 运动学（带缓存优化）
# ==========================
@lru_cache(maxsize=1000)
def missile_position_abs(t_abs: float) -> np.ndarray:
    if t_abs >= T_M_total:
        return np.array([0.0, 0.0, 0.0], dtype=np.float32)
    return M0 + v_M_vec * t_abs

@lru_cache(maxsize=1000)
def drone_unit_and_speed_theta(theta: float, v_d: float):
    u_x = np.cos(theta)
    u_y = np.sin(theta)
    u = np.array([u_x, u_y, 0.0], dtype=np.float32)  # Z 坐标
   保持不变
    return u, v_d

@lru_cache(maxsize=1000)
def drone_position_abs_theta(t_abs: float, theta: float, v_d:
   float) -> np.ndarray:
    u, vd = drone_unit_and_speed_theta(theta, v_d)
    return F0 + u * vd * t_abs

```

```python
@lru_cache(maxsize=1000)
def drop_point_theta(theta: float, v_d: float, t1: float) -> np.ndarray:
    P = drone_position_abs_theta(t1, theta, v_d)
    return np.array([P[0], P[1], F0[2]], dtype=np.float32)

@lru_cache(maxsize=1000)
def explosion_point_theta(theta: float, v_d: float, t1: float, tau_b: float) -> Tuple[float, np.ndarray]:
    t_expl_abs = t1 + tau_b
    P_h = drone_position_abs_theta(t_expl_abs, theta, v_d)
    P_e = np.array([P_h[0], P_h[1], F0[2]], dtype=np.float32)
    return t_expl_abs, P_e

@lru_cache(maxsize=1000)
def smoke_center_abs(t_abs: float, t_expl_abs: float, P_expl_x: float, P_expl_y: float, P_expl_z: float):
    tau = t_abs - t_expl_abs
    if tau < 0.0 or tau > valid_after_explosion:
        return None
    return np.array([P_expl_x, P_expl_y, P_expl_z - sink_speed * tau], dtype=np.float32)

# ==========================
# 目标几何采样（带缓存和单精度优化）
# ==========================
_TARGET_POINTS_CACHE = None

def sample_cylinder_surface(radius, height, num_points_per_circle=360, num_height_samples=50):
    global _TARGET_POINTS_CACHE
    if _TARGET_POINTS_CACHE is not None:
        return _TARGET_POINTS_CACHE

    pts = []
```

```
    angles = np.linspace(0, 2*np.pi, num_points_per_circle,
   endpoint=False)
    heights = np.linspace(0, height, num_height_samples)

    # 圆柱侧面采样
    for z in heights:
        for ang in angles:
            x = target_center[0] + radius * np.cos(ang)
            y = target_center[1] + radius * np.sin(ang)
            pts.append([x, y, z])

    # 圆柱顶部边缘采样
    for ang in angles:
        x = target_center[0] + radius * np.cos(ang)
        y = target_center[1] + radius * np.sin(ang)
        pts.append([x, y, height])

    # 顶部中心点
    pts.append([target_center[0], target_center[1], height])

    _TARGET_POINTS_CACHE = np.array(pts, dtype=np.float32)
    return _TARGET_POINTS_CACHE

# 预计算目标点
TARGET_POINTS = sample_cylinder_surface(target_radius,
   target_height)

# ==========================
# 遮蔽仿真（并行优化）
# ==========================
@dataclass
class SimResult:
    intervals: list
    total: float
    t_expl_abs: float
```

```
    P_drop: np.ndarray
    P_expl: np.ndarray
    u_dir: np.ndarray
    v_d: float

_sim_cache = {}

def _params_key(theta, v_d, t1, tau_b, dt, t_after):
    return (round(float(theta), 4), round(float(v_d), 3),
            round(float(t1), 3), round(float(tau_b), 3),
            round(float(dt), 4), round(float(t_after), 3))

def _simulate_single(params, dt, t_after):
    """单个参数的仿真计算"""
    theta, v_d, t1, tau_b = params
    key = _params_key(theta, v_d, t1, tau_b, dt, t_after)
    if key in _sim_cache:
        return _sim_cache[key]

    P_drop = drop_point_theta(theta, v_d, t1)
    t_expl_abs, P_expl = explosion_point_theta(theta, v_d, t1,
    tau_b)
    u_dir, _ = drone_unit_and_speed_theta(theta, v_d)

    if P_expl[2] < 0.0:
        res = SimResult([], 0.0, t_expl_abs, P_drop, P_expl,
   u_dir, v_d)
        _sim_cache[key] = res
        return res

    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.
   uint8)
```

```
153
154     for j, tau in enumerate(times_rel):
155         t_abs = t_expl_abs + tau
156         M = missile_position_abs(t_abs)
157         C = smoke_center_abs(t_abs, t_expl_abs, float(P_expl
    [0]), float(P_expl[1]), float(P_expl[2]))
158         if C is None:
159             continue
160
161         # 向量化计算遮蔽
162         TM = TARGET_POINTS - M
163         CM = C - M
164         dot_product = np.einsum('ij,j->i', TM, CM, optimize=
    True)
165         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
166         lam = dot_product / TM_norm2
167         lam = np.clip(lam, 0.0, 1.0)
168         closest = M + (TM.T * lam).T  # 高效广播计算
169         d2 = np.sum((C - closest)**2, axis=1)
170         obscured[:, j] = (d2 <= R*R).astype(np.uint8)
171
172     full_mask = np.all(obscured == 1, axis=0)
173     idx = np.where(full_mask)[0]
174     intervals = []
175     total = 0.0
176
177     if idx.size:
178         runs = np.split(times_rel[idx], np.where(np.diff(idx)
    != 1)[0]+1)
179         total = sum(r[-1] - r[0] for r in runs)
180         if len(runs) > 0:
181             intervals = [(r[0], r[-1]) for r in runs]
182
183     res = SimResult(intervals, float(total), float(t_expl_abs)
```

```
    , P_drop, P_expl, u_dir, v_d)
184     _sim_cache[key] = res
185     return res
186 
187 def simulate_duration_theta(theta: float, v_d: float, t1:
    float, tau_b: float,
188                             dt: float = 0.01, t_after: float =
     15.0,
189                             want_intervals=True) -> SimResult:
190     return _simulate_single((theta, v_d, t1, tau_b), dt,
    t_after)
191 
192 # 并行计算包装函数（解决pickle问题）
193 def _simulate_wrapper(args):
194     """用于并行计算的包装函数"""
195     params, dt, t_after = args
196     return _simulate_single(params, dt, t_after)
197 
198 def parallel_simulate(params_list, dt, t_after):
199     """并行计算多个参数的仿真结果"""
200     # 准备参数列表
201     args_list = [(p, dt, t_after) for p in params_list]
202 
203     # 根据CPU核心数确定工作进程数
204     num_workers = min(multiprocessing.cpu_count(), len(
    params_list))
205 
206     with ProcessPoolExecutor(max_workers=num_workers) as
    executor:
207         results = list(tqdm(executor.map(_simulate_wrapper,
    args_list),
208                             total=len(args_list),
209                             desc="并行计算进度"))
210     return results
211 
```

```
212 # ==========================
213 # WOA 边界与个体生成
214 # ==========================
215 @dataclass
216 class BoundsTheta:
217     theta: tuple = (0.0, 2*np.pi)
218     vd: tuple = (70.0, 140.0)
219     t1: tuple = (0.0, 40.0)
220     tb: tuple = (0.0, 40.0)
221 
222 BOUNDS_THETA = BoundsTheta()
223 
224 def random_individual_theta(rng: np.random.Generator) -> np.
    ndarray:
225     theta = rng.uniform(*BOUNDS_THETA.theta)
226     vd = rng.uniform(*BOUNDS_THETA.vd)
227     t1 = rng.uniform(*BOUNDS_THETA.t1)
228     tb = rng.uniform(*BOUNDS_THETA.tb)
229     return np.array([theta, vd, t1, tb], dtype=np.float32)
230 
231 def clip_to_bounds_theta(x: np.ndarray) -> np.ndarray:
232     x[0] = np.clip(x[0], *BOUNDS_THETA.theta)
233     x[1] = np.clip(x[1], *BOUNDS_THETA.vd)
234     x[2] = np.clip(x[2], *BOUNDS_THETA.t1)
235     x[3] = np.clip(x[3], *BOUNDS_THETA.tb)
236     return x
237 
238 def gaussian_mutation_theta(x, rng=None, rate=0.8, scale=0.1):
239     ranges = np.array([BOUNDS_THETA.theta[1]-BOUNDS_THETA.
    theta[0],
240                        BOUNDS_THETA.vd[1]-BOUNDS_THETA.vd[0],
241                        BOUNDS_THETA.t1[1]-BOUNDS_THETA.t1[0],
242                        BOUNDS_THETA.tb[1]-BOUNDS_THETA.tb[0]],
     dtype=np.float32)
243     for i in range(len(x)):
```

```
        if rng.random() < rate:
            x[i] += rng.normal(0.0, scale * ranges[i])
    return clip_to_bounds_theta(x)

# ==========================
# 两阶段鲸鱼优化算法（并行优化）
# ==========================
def enhanced_whale_optimization_two_stage_theta_all_best(
        pop_size_coarse=40,  # 减少粗搜索种群大小
        iters_coarse=30,     # 减少粗搜索迭代次数
        dt_coarse=0.05,      # 增大粗搜索时间步长
        t_after_coarse=10.0, # 缩短粗搜索时间范围
        pop_size_refine=20,  # 减少精搜索种群大小
        iters_refine=30,     # 减少精搜索迭代次数
        dt_refine=0.01,      # 适度增大精搜索时间步长
        t_after_refine=15.0, # 适度缩短精搜索时间范围
        a_max=2.5,
        local_prob=0.35,
        restart_prob=0.04,
        stagnation_limit=5,  # 更早终止
        punish_scale=0.25,
        seed=2025,
        verbose=True):

    rng = np.random.default_rng(seed)
    all_global_best = []

    def run_woa(pop_size, iterations, dt, t_after, init_pop=None):
        nonlocal rng, a_max, local_prob, restart_prob, stagnation_limit, punish_scale, all_global_best

        if init_pop is None:
            pop = np.array([random_individual_theta(rng) for _ in range(pop_size)], dtype=np.float32)
```

```
276         else:
277             pop = [p.copy() for p in init_pop]
278             while len(pop) < pop_size:
279                 pop.append(random_individual_theta(rng))
280             pop = np.array(pop, dtype=np.float32)
281 
282         # 并行评估初始种群
283         results = parallel_simulate(pop, dt, t_after)
284         fitness = np.array([r.total for r in results], dtype=
    np.float32)
285         best_idx = int(np.argmax(fitness))
286         best_params = pop[best_idx].copy()
287         best_sim = results[best_idx]
288         best_val = best_sim.total
289         stagnation_counter = 0
290         all_global_best.append((best_val, best_params.copy(),
    best_sim))
291 
292         if verbose:
293             print(f"[WOA start] dt={dt:.4f} iters={iterations}
     pop={pop_size} initial_best={best_val:.4f}")
294 
295         for it in range(1, iterations+1):
296             a = a_max * (1 - (it/iterations)**0.5)
297 
298             # 随机重启
299             if rng.random() < restart_prob:
300                 replace_idx = rng.choice(pop_size, size=max(1,
     int(pop_size*0.08)), replace=False)
301                 for idx in replace_idx:
302                     pop[idx] = random_individual_theta(rng)
303                 # 并行更新被替换的个体
304                 partial_results = parallel_simulate(pop[
    replace_idx], dt, t_after)
305                 for i, idx in enumerate(replace_idx):
```

```
                    fitness[idx] = partial_results[i].total
                    if partial_results[i].total > best_val:
                        best_val = partial_results[i].total
                        best_params = pop[idx].copy()
                        best_sim = partial_results[i]
                        stagnation_counter = 0
                        all_global_best.append((best_val,
best_params.copy(), best_sim))

            # WOA 更新
            for i in range(pop_size):
                r1, r2 = rng.random(), rng.random()
                A = 2*a*r1 - a
                C = 2*r2
                p = rng.random()

                if p < 0.5:
                    if abs(A) < 1.0:
                        D = abs(C*best_params - pop[i])
                        pop[i] = best_params - A * D
                    else:
                        rand_idx = rng.integers(0, pop_size)
                        X_rand = pop[rand_idx]
                        D = abs(C*X_rand - pop[i])
                        pop[i] = X_rand - A * D
                else:
                    D = abs(best_params - pop[i])
                    b = 1
                    l = rng.uniform(-1,1)
                    pop[i] = D * np.exp(b*l) * np.cos(2*np.pi*
l) + best_params

                if rng.random() < local_prob:
                    pop[i] = gaussian_mutation_theta(pop[i],
rng=rng, rate=0.75, scale=0.12)
```

```
338                 pop[i] = clip_to_bounds_theta(pop[i])
339 
340             # 并行评估新种群
341             results = parallel_simulate(pop, dt, t_after)
342             fitness = np.array([r.total for r in results],
    dtype=np.float32)
343 
344             cur_best_idx = int(np.argmax(fitness))
345             cur_best_val = fitness[cur_best_idx]
346 
347             if cur_best_val > best_val:
348                 best_params = pop[cur_best_idx].copy()
349                 best_sim = results[cur_best_idx]
350                 best_val = best_sim.total
351                 stagnation_counter = 0
352                 all_global_best.append((best_val, best_params.
    copy(), best_sim))
353                 if verbose:
354                     print(f"[WOA] iter={it} new_best={best_val
    :.4f} params={np.round(best_params,3)}")
355             else:
356                 stagnation_counter += 1
357 
358             # 提前终止检查
359             if stagnation_counter >= stagnation_limit:
360                 if verbose:
361                     print(f"Early termination at iteration {it
    } due to stagnation")
362                 break
363 
364         return best_params, best_sim, pop
365 
366     # -------- coarse stage --------
367     best_coarse_params, best_coarse_sim, pop_coarse = run_woa(
368         pop_size_coarse, iters_coarse, dt_coarse,
```

```
    t_after_coarse)
369
370     # -------- refine stage --------
371     rng_local = np.random.default_rng(seed+1)
372     init_pop_refine = [best_coarse_params.copy()]
373     for _ in range(max(5, pop_size_refine-5)):
374         pert = gaussian_mutation_theta(best_coarse_params.copy
    (), rng=rng_local, rate=0.9, scale=0.08)
375         init_pop_refine.append(pert)
376     while len(init_pop_refine) < pop_size_refine:
377         init_pop_refine.append(random_individual_theta(
    rng_local))
378
379     best_refine_params, best_refine_sim, _ = run_woa(
380         pop_size_refine, iters_refine, dt_refine,
    t_after_refine, init_pop=init_pop_refine)
381
382     # 取所有最优结果
383     max_total = max(g[0] for g in all_global_best)
384     best_all = [(p, sim) for val, p, sim in all_global_best if
     val == max_total]
385
386     if verbose:
387         print(f"[All best solutions found: {len(best_all)}]
    total={max_total:.4f}")
388
389     return best_all
390
391 # =========================
392 # 输出 & 可视化（优化后）
393 # =========================
394 def print_result_table_theta(sim: SimResult, grenade_id=1):
395     u = sim.u_dir
396     theta = np.arctan2(u[1], u[0])
397     data = [
```

```
        ["无人机运动方向 (水平平面 θ)", f"{theta:.3f} rad"],
        ["无人机运动速度 (m/s)", f"{sim.v_d:.3f}"],
        ["烟幕干扰弹编号", f"{grenade_id}"],
        ["烟幕干扰弹投放点 x (m)", f"{sim.P_drop[0]:.2f}"],
        ["烟幕干扰弹投放点 y (m)", f"{sim.P_drop[1]:.2f}"],
        ["烟幕干扰弹投放点 z (m)", f"{sim.P_drop[2]:.2f}"],
        ["烟幕干扰弹起爆点 x (m)", f"{sim.P_expl[0]:.2f}"],
        ["烟幕干扰弹起爆点 y (m)", f"{sim.P_expl[1]:.2f}"],
        ["烟幕干扰弹起爆点 z (m)", f"{sim.P_expl[2]:.2f}"],
        ["有效干扰时长 (s)", f"{sim.total:.3f}"],
    ]

    print("\n=== 优化结果一览（表格） ===")
    print(" | ".join([d[0] for d in data]))
    print(" | ".join([d[1] for d in data]))


def plot_heatmap_theta(sim: SimResult, dt=0.01, t_after=15.0):
    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.
uint8)

    for j, tau in enumerate(times_rel):
        t_abs = sim.t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, sim.t_expl_abs, float(sim.
P_expl[0]), float(sim.P_expl[1]), float(sim.P_expl[2]))
        if C is None:
            continue

        TM = TARGET_POINTS - M
        CM = C - M
        dot_product = np.einsum('ij,j->i', TM, CM, optimize=
```

```
    True)
431         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
432         lam = dot_product / TM_norm2
433         lam = np.clip(lam, 0.0, 1.0)
434         closest = M + (TM.T * lam).T
435         d2 = np.sum((C - closest)**2, axis=1)
436         obscured[:, j] = (d2 <= R*R).astype(np.uint8)
437 
438     plt.figure(figsize=(10, 6))
439     plt.imshow(obscured, aspect='auto', origin='lower', cmap='
    Greens')
440     plt.yticks([0, len(TARGET_POINTS)//2, len(TARGET_POINTS)
    -1], ["底部", "中部", "顶部"])
441     plt.xlabel("时间步")
442     plt.ylabel("目标点区域")
443     plt.title("遮蔽热力图")
444     plt.colorbar(label="是否被遮蔽")
445     plt.tight_layout()
446     plt.show()
447 
448 def plot_target_points():
449     fig = plt.figure(figsize=(10, 6))
450     ax = fig.add_subplot(111, projection='3d')
451     ax.scatter(TARGET_POINTS[:,0], TARGET_POINTS[:,1],
    TARGET_POINTS[:,2], s=5, c='r', alpha=0.6)
452     ax.set_xlabel('X (m)')
453     ax.set_ylabel('Y (m)')
454     ax.set_zlabel('Z (m)')
455     ax.set_title('目标采样点分布')
456     plt.tight_layout()
457     plt.show()
458 
459 # ==========================
460 # 主程序（优化执行流程）
```

```
# ==========================
if __name__ == "__main__":
    print("优化开始，请等待...")
    start_time = time.time()

    # 运行优化算法（使用优化后的参数）
    all_best = enhanced_whale_optimization_two_stage_theta_all_best(
        pop_size_coarse=40,
        iters_coarse=100,
        dt_coarse=0.05,
        t_after_coarse=10.0,
        pop_size_refine=20,
        iters_refine=100,
        dt_refine=0.01,
        t_after_refine=15.0,
        verbose=True
    )

    # 输出结果
    print(f"\n优化完成，耗时: {time.time()-start_time:.2f}秒")
    for idx, (params, sim) in enumerate(all_best[:3]):  # 只显示前3个最优解
        print(f"\n--- 最优方案 {idx+1} ---")
        print_result_table_theta(sim, grenade_id=idx+1)
        if idx == 0:  # 只绘制第一个最优解的热图
            plot_heatmap_theta(sim)
```

Q3.py

```
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from tqdm import tqdm
import time
from typing import Tuple
```

```python
from mpl_toolkits.mplot3d import Axes3D
from concurrent.futures import ProcessPoolExecutor
import multiprocessing
from functools import lru_cache
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# ==========================
# 常量 & 场景参数
# ==========================
g = 9.81

# M1 与 FY1 初始状态
M0 = np.array([20000.0, 0.0, 2000.0], dtype=np.float32)
F0 = np.array([17800.0, 0.0, 1800.0], dtype=np.float32)

# —— 真目标（圆柱）——
target_center = np.array([0.0, 200.0, 0.0], dtype=np.float32)
target_radius = 7.0
target_height = 10.0

# —— 云团参数——
R = 10.0
sink_speed = 3.0
valid_after_explosion = 20.0

# M1 速度向量（固定）
sqrt101 = np.sqrt(101.0)
v_M_vec = 300.0 * np.array([-10.0/sqrt101, 0.0, -1.0/sqrt101], dtype=np.float32)
T_M_total = np.linalg.norm(M0 - np.array([0.0, 0.0, 0.0], dtype=np.float32)) / 300.0

# ==========================
# 运动学（带缓存优化）
```

```python
# ==========================
@lru_cache(maxsize=1000)
def missile_position_abs(t_abs: float) -> np.ndarray:
    if t_abs >= T_M_total:
        return np.array([0.0, 0.0, 0.0], dtype=np.float32)
    return M0 + v_M_vec * t_abs

@lru_cache(maxsize=1000)
def drone_unit_and_speed_theta(theta: float, v_d: float):
    u_x = np.cos(theta)
    u_y = np.sin(theta)
    u = np.array([u_x, u_y, 0.0], dtype=np.float32)  # Z 坐标保持不变
    return u, v_d

@lru_cache(maxsize=1000)
def drone_position_abs_theta(t_abs: float, theta: float, v_d: float) -> np.ndarray:
    u, vd = drone_unit_and_speed_theta(theta, v_d)
    return F0 + u * vd * t_abs

@lru_cache(maxsize=1000)
def drop_point_theta(theta: float, v_d: float, t1: float) -> np.ndarray:
    P = drone_position_abs_theta(t1, theta, v_d)
    return np.array([P[0], P[1], F0[2]], dtype=np.float32)

@lru_cache(maxsize=1000)
def explosion_point_theta(theta: float, v_d: float, t1: float, tau_b: float) -> Tuple[float, np.ndarray]:
    t_expl_abs = t1 + tau_b
    P_h = drone_position_abs_theta(t_expl_abs, theta, v_d)
    P_e = np.array([P_h[0], P_h[1], F0[2]], dtype=np.float32)
    return t_expl_abs, P_e

```

```
@lru_cache(maxsize=1000)
def smoke_center_abs(t_abs: float, t_expl_abs: float, P_expl_x: float, P_expl_y: float, P_expl_z: float):
    tau = t_abs - t_expl_abs
    if tau < 0.0 or tau > valid_after_explosion:
        return None
    return np.array([P_expl_x, P_expl_y, P_expl_z - sink_speed * tau], dtype=np.float32)

# ==========================
# 目标几何采样（带缓存和单精度优化）
# ==========================
_TARGET_POINTS_CACHE = None

def sample_cylinder_surface(radius, height, num_points_per_circle=360, num_height_samples=50):
    global _TARGET_POINTS_CACHE
    if _TARGET_POINTS_CACHE is not None:
        return _TARGET_POINTS_CACHE

    pts = []
    angles = np.linspace(0, 2*np.pi, num_points_per_circle, endpoint=False)
    heights = np.linspace(0, height, num_height_samples)

    # 圆柱侧面采样
    for z in heights:
        for ang in angles:
            x = target_center[0] + radius * np.cos(ang)
            y = target_center[1] + radius * np.sin(ang)
            pts.append([x, y, z])

    # 圆柱顶部边缘采样
    for ang in angles:
        x = target_center[0] + radius * np.cos(ang)
```

```
        y = target_center[1] + radius * np.sin(ang)
        pts.append([x, y, height])

    # 顶部中心点
    pts.append([target_center[0], target_center[1], height])

    _TARGET_POINTS_CACHE = np.array(pts, dtype=np.float32)
    return _TARGET_POINTS_CACHE

# 预计算目标点
TARGET_POINTS = sample_cylinder_surface(target_radius,
   target_height)

# ==========================
# 遮蔽仿真（并行优化）
# ==========================
@dataclass
class SimResult:
    intervals: list
    total: float
    t_expl_abs: float
    P_drop: np.ndarray
    P_expl: np.ndarray
    u_dir: np.ndarray
    v_d: float

_sim_cache = {}

def _params_key(theta, v_d, t1, tau_b, dt, t_after):
    return (round(float(theta), 4), round(float(v_d), 3),
            round(float(t1), 3), round(float(tau_b), 3),
            round(float(dt), 4), round(float(t_after), 3))

def _simulate_single(params, dt, t_after):
    """单个参数的仿真计算"""
```

```
    theta, v_d, t1, tau_b = params
    key = _params_key(theta, v_d, t1, tau_b, dt, t_after)
    if key in _sim_cache:
        return _sim_cache[key]

    P_drop = drop_point_theta(theta, v_d, t1)
    t_expl_abs, P_expl = explosion_point_theta(theta, v_d, t1,
    tau_b)
    u_dir, _ = drone_unit_and_speed_theta(theta, v_d)

    if P_expl[2] < 0.0:
        res = SimResult([], 0.0, t_expl_abs, P_drop, P_expl,
u_dir, v_d)
        _sim_cache[key] = res
        return res

    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.
uint8)

    for j, tau in enumerate(times_rel):
        t_abs = t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, t_expl_abs, float(P_expl
[0]), float(P_expl[1]), float(P_expl[2]))
        if C is None:
            continue

        # 向量化计算遮蔽
        TM = TARGET_POINTS - M
        CM = C - M
        dot_product = np.einsum('ij,j->i', TM, CM, optimize=
True)
```

```
165         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
166         lam = dot_product / TM_norm2
167         lam = np.clip(lam, 0.0, 1.0)
168         closest = M + (TM.T * lam).T  # 高效广播计算
169         d2 = np.sum((C - closest)**2, axis=1)
170         obscured[:, j] = (d2 <= R*R).astype(np.uint8)
171 
172     full_mask = np.all(obscured == 1, axis=0)
173     idx = np.where(full_mask)[0]
174     intervals = []
175     total = 0.0
176 
177     if idx.size:
178         runs = np.split(times_rel[idx], np.where(np.diff(idx)
    != 1)[0]+1)
179         total = sum(r[-1] - r[0] for r in runs)
180         if len(runs) > 0:
181             intervals = [(r[0], r[-1]) for r in runs]
182 
183     res = SimResult(intervals, float(total), float(t_expl_abs)
    , P_drop, P_expl, u_dir, v_d)
184     _sim_cache[key] = res
185     return res
186 
187 def simulate_duration_theta(theta: float, v_d: float, t1:
    float, tau_b: float,
188                             dt: float = 0.01, t_after: float =
     15.0,
189                             want_intervals=True) -> SimResult:
190     return _simulate_single((theta, v_d, t1, tau_b), dt,
    t_after)
191 
192 # 并行计算包装函数（解决pickle问题）
193 def _simulate_wrapper(args):
```

```
194     """用于并行计算的包装函数"""
195     params, dt, t_after = args
196     return _simulate_single(params, dt, t_after)
197 
198 def parallel_simulate(params_list, dt, t_after):
199     """并行计算多个参数的仿真结果"""
200     # 准备参数列表
201     args_list = [(p, dt, t_after) for p in params_list]
202 
203     # 根据CPU核心数确定工作进程数
204     num_workers = min(multiprocessing.cpu_count(), len(
    params_list))
205 
206     with ProcessPoolExecutor(max_workers=num_workers) as
    executor:
207         results = list(tqdm(executor.map(_simulate_wrapper,
    args_list),
208                             total=len(args_list),
209                             desc="并行计算进度"))
210     return results
211 
212 # =========================
213 # WOA 边界与个体生成
214 # =========================
215 @dataclass
216 class BoundsTheta:
217     theta: tuple = (0.0, 2*np.pi)
218     vd: tuple = (70.0, 140.0)
219     t1: tuple = (0.0, 40.0)
220     tb: tuple = (0.0, 40.0)
221 
222 BOUNDS_THETA = BoundsTheta()
223 
224 def random_individual_theta(rng: np.random.Generator) -> np.
    ndarray:
```

```
225     theta = rng.uniform(*BOUNDS_THETA.theta)
226     vd = rng.uniform(*BOUNDS_THETA.vd)
227     t1 = rng.uniform(*BOUNDS_THETA.t1)
228     tb = rng.uniform(*BOUNDS_THETA.tb)
229     return np.array([theta, vd, t1, tb], dtype=np.float32)
230 
231 def clip_to_bounds_theta(x: np.ndarray) -> np.ndarray:
232     x[0] = np.clip(x[0], *BOUNDS_THETA.theta)
233     x[1] = np.clip(x[1], *BOUNDS_THETA.vd)
234     x[2] = np.clip(x[2], *BOUNDS_THETA.t1)
235     x[3] = np.clip(x[3], *BOUNDS_THETA.tb)
236     return x
237 
238 def gaussian_mutation_theta(x, rng=None, rate=0.8, scale=0.1):
239     ranges = np.array([BOUNDS_THETA.theta[1]-BOUNDS_THETA.
    theta[0],
240                        BOUNDS_THETA.vd[1]-BOUNDS_THETA.vd[0],
241                        BOUNDS_THETA.t1[1]-BOUNDS_THETA.t1[0],
242                        BOUNDS_THETA.tb[1]-BOUNDS_THETA.tb[0]],
     dtype=np.float32)
243     for i in range(len(x)):
244         if rng.random() < rate:
245             x[i] += rng.normal(0.0, scale * ranges[i])
246     return clip_to_bounds_theta(x)
247 
248 # ==========================
249 # 两阶段鲸鱼优化算法（并行优化）
250 # ==========================
251 def enhanced_whale_optimization_two_stage_theta_all_best(
252         pop_size_coarse=40,  # 减少粗搜索种群大小
253         iters_coarse=30,     # 减少粗搜索迭代次数
254         dt_coarse=0.05,      # 增大粗搜索时间步长
255         t_after_coarse=10.0, # 缩短粗搜索时间范围
256         pop_size_refine=20,  # 减少精搜索种群大小
257         iters_refine=30,     # 减少精搜索迭代次数
```

```
        dt_refine=0.01,       # 适度增大精搜索时间步长
        t_after_refine=15.0, # 适度缩短精搜索时间范围
        a_max=2.5,
        local_prob=0.35,
        restart_prob=0.04,
        stagnation_limit=5,  # 更早终止
        punish_scale=0.25,
        seed=2025,
        verbose=True):

    rng = np.random.default_rng(seed)
    all_global_best = []

    def run_woa(pop_size, iterations, dt, t_after, init_pop=None):
        nonlocal rng, a_max, local_prob, restart_prob, stagnation_limit, punish_scale, all_global_best

        if init_pop is None:
            pop = np.array([random_individual_theta(rng) for _ in range(pop_size)], dtype=np.float32)
        else:
            pop = [p.copy() for p in init_pop]
            while len(pop) < pop_size:
                pop.append(random_individual_theta(rng))
            pop = np.array(pop, dtype=np.float32)

        # 并行评估初始种群
        results = parallel_simulate(pop, dt, t_after)
        fitness = np.array([r.total for r in results], dtype=np.float32)
        best_idx = int(np.argmax(fitness))
        best_params = pop[best_idx].copy()
        best_sim = results[best_idx]
        best_val = best_sim.total
```

```
289         stagnation_counter = 0
290         all_global_best.append((best_val, best_params.copy(),
    best_sim))
291 
292         if verbose:
293             print(f"[WOA start] dt={dt:.4f} iters={iterations}
     pop={pop_size} initial_best={best_val:.4f}")
294 
295         for it in range(1, iterations+1):
296             a = a_max * (1 - (it/iterations)**0.5)
297 
298             # 随机重启
299             if rng.random() < restart_prob:
300                 replace_idx = rng.choice(pop_size, size=max(1,
     int(pop_size*0.08)), replace=False)
301                 for idx in replace_idx:
302                     pop[idx] = random_individual_theta(rng)
303                 # 并行更新被替换的个体
304                 partial_results = parallel_simulate(pop[
    replace_idx], dt, t_after)
305                 for i, idx in enumerate(replace_idx):
306                     fitness[idx] = partial_results[i].total
307                     if partial_results[i].total > best_val:
308                         best_val = partial_results[i].total
309                         best_params = pop[idx].copy()
310                         best_sim = partial_results[i]
311                         stagnation_counter = 0
312                         all_global_best.append((best_val,
    best_params.copy(), best_sim))
313 
314             # WOA 更新
315             for i in range(pop_size):
316                 r1, r2 = rng.random(), rng.random()
317                 A = 2*a*r1 - a
318                 C = 2*r2
```

```
                p = rng.random()

                if p < 0.5:
                    if abs(A) < 1.0:
                        D = abs(C*best_params - pop[i])
                        pop[i] = best_params - A * D
                    else:
                        rand_idx = rng.integers(0, pop_size)
                        X_rand = pop[rand_idx]
                        D = abs(C*X_rand - pop[i])
                        pop[i] = X_rand - A * D
                else:
                    D = abs(best_params - pop[i])
                    b = 1
                    l = rng.uniform(-1,1)
                    pop[i] = D * np.exp(b*l) * np.cos(2*np.pi*
l) + best_params

                if rng.random() < local_prob:
                    pop[i] = gaussian_mutation_theta(pop[i],
rng=rng, rate=0.75, scale=0.12)
                pop[i] = clip_to_bounds_theta(pop[i])

            # 并行评估新种群
            results = parallel_simulate(pop, dt, t_after)
            fitness = np.array([r.total for r in results],
dtype=np.float32)

            cur_best_idx = int(np.argmax(fitness))
            cur_best_val = fitness[cur_best_idx]

            if cur_best_val > best_val:
                best_params = pop[cur_best_idx].copy()
                best_sim = results[cur_best_idx]
                best_val = best_sim.total
```

```
                stagnation_counter = 0
                all_global_best.append((best_val, best_params.
    copy(), best_sim))
                if verbose:
                    print(f"[WOA] iter={it} new_best={best_val
    :.4f} params={np.round(best_params,3)}")
            else:
                stagnation_counter += 1

            # 提前终止检查
            if stagnation_counter >= stagnation_limit:
                if verbose:
                    print(f"Early termination at iteration {it
    } due to stagnation")
                break

        return best_params, best_sim, pop

    # -------- coarse stage --------
    best_coarse_params, best_coarse_sim, pop_coarse = run_woa(
        pop_size_coarse, iters_coarse, dt_coarse,
    t_after_coarse)

    # -------- refine stage --------
    rng_local = np.random.default_rng(seed+1)
    init_pop_refine = [best_coarse_params.copy()]
    for _ in range(max(5, pop_size_refine-5)):
        pert = gaussian_mutation_theta(best_coarse_params.copy
    (), rng=rng_local, rate=0.9, scale=0.08)
        init_pop_refine.append(pert)
    while len(init_pop_refine) < pop_size_refine:
        init_pop_refine.append(random_individual_theta(
    rng_local))

    best_refine_params, best_refine_sim, _ = run_woa(
```

```
380         pop_size_refine, iters_refine, dt_refine,
    t_after_refine, init_pop=init_pop_refine)
381
382     # 取所有最优结果
383     max_total = max(g[0] for g in all_global_best)
384     best_all = [(p, sim) for val, p, sim in all_global_best if
    val == max_total]
385
386     if verbose:
387         print(f"[All best solutions found: {len(best_all)}]
    total={max_total:.4f}")
388
389     return best_all
390
391 # ==========================
392 # 输出 & 可视化（优化后）
393 # ==========================
394 def print_result_table_theta(sim: SimResult, grenade_id=1):
395     u = sim.u_dir
396     theta = np.arctan2(u[1], u[0])
397     data = [
398         ["无人机运动方向（水平平面 θ)", f"{theta:.3f} rad"],
399         ["无人机运动速度 (m/s)", f"{sim.v_d:.3f}"],
400         ["烟幕干扰弹编号", f"{grenade_id}"],
401         ["烟幕干扰弹投放点 x (m)", f"{sim.P_drop[0]:.2f}"],
402         ["烟幕干扰弹投放点 y (m)", f"{sim.P_drop[1]:.2f}"],
403         ["烟幕干扰弹投放点 z (m)", f"{sim.P_drop[2]:.2f}"],
404         ["烟幕干扰弹起爆点 x (m)", f"{sim.P_expl[0]:.2f}"],
405         ["烟幕干扰弹起爆点 y (m)", f"{sim.P_expl[1]:.2f}"],
406         ["烟幕干扰弹起爆点 z (m)", f"{sim.P_expl[2]:.2f}"],
407         ["有效干扰时长 (s)", f"{sim.total:.3f}"],
408     ]
409
410     print("\n=== 优化结果一览（表格） ===")
411     print(" | ".join([d[0] for d in data]))
```

```
    print(" | ".join([d[1] for d in data]))


def plot_heatmap_theta(sim: SimResult, dt=0.01, t_after=15.0):
    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.uint8)

    for j, tau in enumerate(times_rel):
        t_abs = sim.t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, sim.t_expl_abs, float(sim.P_expl[0]), float(sim.P_expl[1]), float(sim.P_expl[2]))
        if C is None:
            continue

        TM = TARGET_POINTS - M
        CM = C - M
        dot_product = np.einsum('ij,j->i', TM, CM, optimize=True)
        TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True)
        lam = dot_product / TM_norm2
        lam = np.clip(lam, 0.0, 1.0)
        closest = M + (TM.T * lam).T
        d2 = np.sum((C - closest)**2, axis=1)
        obscured[:, j] = (d2 <= R*R).astype(np.uint8)

    plt.figure(figsize=(10, 6))
    plt.imshow(obscured, aspect='auto', origin='lower', cmap='Greens')
    plt.yticks([0, len(TARGET_POINTS)//2, len(TARGET_POINTS)-1], ["底部", "中部", "顶部"])
```

```
441     plt.xlabel("时间步")
442     plt.ylabel("目标点区域")
443     plt.title("遮蔽热力图")
444     plt.colorbar(label="是否被遮蔽")
445     plt.tight_layout()
446     plt.show()
447 
448 def plot_target_points():
449     fig = plt.figure(figsize=(10, 6))
450     ax = fig.add_subplot(111, projection='3d')
451     ax.scatter(TARGET_POINTS[:,0], TARGET_POINTS[:,1],
    TARGET_POINTS[:,2], s=5, c='r', alpha=0.6)
452     ax.set_xlabel('X (m)')
453     ax.set_ylabel('Y (m)')
454     ax.set_zlabel('Z (m)')
455     ax.set_title('目标采样点分布')
456     plt.tight_layout()
457     plt.show()
458 
459 # ==========================
460 # 主程序（优化执行流程）
461 # ==========================
462 if __name__ == "__main__":
463     print("优化开始，请等待...")
464     start_time = time.time()
465 
466     # 运行优化算法（使用优化后的参数）
467     all_best =
    enhanced_whale_optimization_two_stage_theta_all_best(
468         pop_size_coarse=40,
469         iters_coarse=100,
470         dt_coarse=0.05,
471         t_after_coarse=10.0,
472         pop_size_refine=20,
473         iters_refine=100,
```

```
        dt_refine=0.01,
        t_after_refine=15.0,
        verbose=True
    )

    # 输出结果
    print(f"\n优化完成，耗时: {time.time()-start_time:.2f}秒")
    for idx, (params, sim) in enumerate(all_best[:3]):  # 只显示前3个最优解
        print(f"\n--- 最优方案 {idx+1} ---")
        print_result_table_theta(sim, grenade_id=idx+1)
        if idx == 0:  # 只绘制第一个最优解的热图
            plot_heatmap_theta(sim)
```

Q4.py

```
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from tqdm import tqdm
import time
from typing import Tuple
from mpl_toolkits.mplot3d import Axes3D
from concurrent.futures import ProcessPoolExecutor
import multiprocessing
from functools import lru_cache
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# ========================
# 常量 & 场景参数
# ========================
g = 9.81

# M1 与 FY1 初始状态
M0 = np.array([20000.0, 0.0, 2000.0], dtype=np.float32)
```

```python
F0 = np.array([17800.0, 0.0, 1800.0], dtype=np.float32)

# —— 真目标（圆柱）——
target_center = np.array([0.0, 200.0, 0.0], dtype=np.float32)
target_radius = 7.0
target_height = 10.0

# —— 云团参数——
R = 10.0
sink_speed = 3.0
valid_after_explosion = 20.0

# M1 速度向量（固定）
sqrt101 = np.sqrt(101.0)
v_M_vec = 300.0 * np.array([-10.0/sqrt101, 0.0, -1.0/sqrt101],
    dtype=np.float32)
T_M_total = np.linalg.norm(M0 - np.array([0.0, 0.0, 0.0],
   dtype=np.float32)) / 300.0

# ==========================
# 运动学（带缓存优化）
# ==========================
@lru_cache(maxsize=1000)
def missile_position_abs(t_abs: float) -> np.ndarray:
    if t_abs >= T_M_total:
        return np.array([0.0, 0.0, 0.0], dtype=np.float32)
    return M0 + v_M_vec * t_abs

@lru_cache(maxsize=1000)
def drone_unit_and_speed_theta(theta: float, v_d: float):
    u_x = np.cos(theta)
    u_y = np.sin(theta)
    u = np.array([u_x, u_y, 0.0], dtype=np.float32)  # Z 坐标
   保持不变
    return u, v_d
```

```
52
53  @lru_cache(maxsize=1000)
54  def drone_position_abs_theta(t_abs: float, theta: float, v_d:
        float) -> np.ndarray:
55      u, vd = drone_unit_and_speed_theta(theta, v_d)
56      return F0 + u * vd * t_abs
57
58  @lru_cache(maxsize=1000)
59  def drop_point_theta(theta: float, v_d: float, t1: float) ->
        np.ndarray:
60      P = drone_position_abs_theta(t1, theta, v_d)
61      return np.array([P[0], P[1], F0[2]], dtype=np.float32)
62
63  @lru_cache(maxsize=1000)
64  def explosion_point_theta(theta: float, v_d: float, t1: float,
        tau_b: float) -> Tuple[float, np.ndarray]:
65      t_expl_abs = t1 + tau_b
66      P_h = drone_position_abs_theta(t_expl_abs, theta, v_d)
67      P_e = np.array([P_h[0], P_h[1], F0[2]], dtype=np.float32)
68      return t_expl_abs, P_e
69
70  @lru_cache(maxsize=1000)
71  def smoke_center_abs(t_abs: float, t_expl_abs: float, P_expl_x
        : float, P_expl_y: float, P_expl_z: float):
72      tau = t_abs - t_expl_abs
73      if tau < 0.0 or tau > valid_after_explosion:
74          return None
75      return np.array([P_expl_x, P_expl_y, P_expl_z - sink_speed
        * tau], dtype=np.float32)
76
77  # ==========================
78  # 目标几何采样（带缓存和单精度优化）
79  # ==========================
80  _TARGET_POINTS_CACHE = None
81
```

```
82 def sample_cylinder_surface(radius, height,
   num_points_per_circle=360, num_height_samples=50):
83     global _TARGET_POINTS_CACHE
84     if _TARGET_POINTS_CACHE is not None:
85         return _TARGET_POINTS_CACHE
86 
87     pts = []
88     angles = np.linspace(0, 2*np.pi, num_points_per_circle,
   endpoint=False)
89     heights = np.linspace(0, height, num_height_samples)
90 
91     # 圆柱侧面采样
92     for z in heights:
93         for ang in angles:
94             x = target_center[0] + radius * np.cos(ang)
95             y = target_center[1] + radius * np.sin(ang)
96             pts.append([x, y, z])
97 
98     # 圆柱顶部边缘采样
99     for ang in angles:
100        x = target_center[0] + radius * np.cos(ang)
101        y = target_center[1] + radius * np.sin(ang)
102        pts.append([x, y, height])
103 
104    # 顶部中心点
105    pts.append([target_center[0], target_center[1], height])
106 
107    _TARGET_POINTS_CACHE = np.array(pts, dtype=np.float32)
108    return _TARGET_POINTS_CACHE
109 
110 # 预计算目标点
111 TARGET_POINTS = sample_cylinder_surface(target_radius,
   target_height)
112 
113 # ==========================
```

```python
# 遮蔽仿真（并行优化）
# ==========================
@dataclass
class SimResult:
    intervals: list
    total: float
    t_expl_abs: float
    P_drop: np.ndarray
    P_expl: np.ndarray
    u_dir: np.ndarray
    v_d: float

_sim_cache = {}

def _params_key(theta, v_d, t1, tau_b, dt, t_after):
    return (round(float(theta), 4), round(float(v_d), 3),
            round(float(t1), 3), round(float(tau_b), 3),
            round(float(dt), 4), round(float(t_after), 3))

def _simulate_single(params, dt, t_after):
    """单个参数的仿真计算"""
    theta, v_d, t1, tau_b = params
    key = _params_key(theta, v_d, t1, tau_b, dt, t_after)
    if key in _sim_cache:
        return _sim_cache[key]

    P_drop = drop_point_theta(theta, v_d, t1)
    t_expl_abs, P_expl = explosion_point_theta(theta, v_d, t1,
    tau_b)
    u_dir, _ = drone_unit_and_speed_theta(theta, v_d)

    if P_expl[2] < 0.0:
        res = SimResult([], 0.0, t_expl_abs, P_drop, P_expl,
   u_dir, v_d)
        _sim_cache[key] = res
```

```
        return res

    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.uint8)

    for j, tau in enumerate(times_rel):
        t_abs = t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, t_expl_abs, float(P_expl[0]), float(P_expl[1]), float(P_expl[2]))
        if C is None:
            continue

        # 向量化计算遮蔽
        TM = TARGET_POINTS - M
        CM = C - M
        dot_product = np.einsum('ij,j->i', TM, CM, optimize=True)
        TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True)
        lam = dot_product / TM_norm2
        lam = np.clip(lam, 0.0, 1.0)
        closest = M + (TM.T * lam).T  # 高效广播计算
        d2 = np.sum((C - closest)**2, axis=1)
        obscured[:, j] = (d2 <= R*R).astype(np.uint8)

    full_mask = np.all(obscured == 1, axis=0)
    idx = np.where(full_mask)[0]
    intervals = []
    total = 0.0

    if idx.size:
```

```
178         runs = np.split(times_rel[idx], np.where(np.diff(idx)
    != 1)[0]+1)
179         total = sum(r[-1] - r[0] for r in runs)
180         if len(runs) > 0:
181             intervals = [(r[0], r[-1]) for r in runs]
182 
183     res = SimResult(intervals, float(total), float(t_expl_abs)
    , P_drop, P_expl, u_dir, v_d)
184     _sim_cache[key] = res
185     return res
186 
187 def simulate_duration_theta(theta: float, v_d: float, t1:
    float, tau_b: float,
188                             dt: float = 0.01, t_after: float =
     15.0,
189                             want_intervals=True) -> SimResult:
190     return _simulate_single((theta, v_d, t1, tau_b), dt,
    t_after)
191 
192 # 并行计算包装函数（解决pickle问题）
193 def _simulate_wrapper(args):
194     """用于并行计算的包装函数"""
195     params, dt, t_after = args
196     return _simulate_single(params, dt, t_after)
197 
198 def parallel_simulate(params_list, dt, t_after):
199     """并行计算多个参数的仿真结果"""
200     # 准备参数列表
201     args_list = [(p, dt, t_after) for p in params_list]
202 
203     # 根据CPU核心数确定工作进程数
204     num_workers = min(multiprocessing.cpu_count(), len(
    params_list))
205 
206     with ProcessPoolExecutor(max_workers=num_workers) as
```

```
    executor:
207         results = list(tqdm(executor.map(_simulate_wrapper,
    args_list),
208                             total=len(args_list),
209                             desc="并行计算进度"))
210     return results
211
212 # ==========================
213 # WOA 边界与个体生成
214 # ==========================
215 @dataclass
216 class BoundsTheta:
217     theta: tuple = (0.0, 2*np.pi)
218     vd: tuple = (70.0, 140.0)
219     t1: tuple = (0.0, 40.0)
220     tb: tuple = (0.0, 40.0)
221
222 BOUNDS_THETA = BoundsTheta()
223
224 def random_individual_theta(rng: np.random.Generator) -> np.
    ndarray:
225     theta = rng.uniform(*BOUNDS_THETA.theta)
226     vd = rng.uniform(*BOUNDS_THETA.vd)
227     t1 = rng.uniform(*BOUNDS_THETA.t1)
228     tb = rng.uniform(*BOUNDS_THETA.tb)
229     return np.array([theta, vd, t1, tb], dtype=np.float32)
230
231 def clip_to_bounds_theta(x: np.ndarray) -> np.ndarray:
232     x[0] = np.clip(x[0], *BOUNDS_THETA.theta)
233     x[1] = np.clip(x[1], *BOUNDS_THETA.vd)
234     x[2] = np.clip(x[2], *BOUNDS_THETA.t1)
235     x[3] = np.clip(x[3], *BOUNDS_THETA.tb)
236     return x
237
238 def gaussian_mutation_theta(x, rng=None, rate=0.8, scale=0.1):
```

```
    ranges = np.array([BOUNDS_THETA.theta[1]-BOUNDS_THETA.theta[0],
                       BOUNDS_THETA.vd[1]-BOUNDS_THETA.vd[0],
                       BOUNDS_THETA.t1[1]-BOUNDS_THETA.t1[0],
                       BOUNDS_THETA.tb[1]-BOUNDS_THETA.tb[0]], dtype=np.float32)
    for i in range(len(x)):
        if rng.random() < rate:
            x[i] += rng.normal(0.0, scale * ranges[i])
    return clip_to_bounds_theta(x)

# ==========================
# 两阶段鲸鱼优化算法（并行优化）
# ==========================
def enhanced_whale_optimization_two_stage_theta_all_best(
        pop_size_coarse=40,  # 减少粗搜索种群大小
        iters_coarse=30,     # 减少粗搜索迭代次数
        dt_coarse=0.05,      # 增大粗搜索时间步长
        t_after_coarse=10.0, # 缩短粗搜索时间范围
        pop_size_refine=20,  # 减少精搜索种群大小
        iters_refine=30,     # 减少精搜索迭代次数
        dt_refine=0.01,      # 适度增大精搜索时间步长
        t_after_refine=15.0, # 适度缩短精搜索时间范围
        a_max=2.5,
        local_prob=0.35,
        restart_prob=0.04,
        stagnation_limit=5,  # 更早终止
        punish_scale=0.25,
        seed=2025,
        verbose=True):

    rng = np.random.default_rng(seed)
    all_global_best = []

    def run_woa(pop_size, iterations, dt, t_after, init_pop=
```

```
    None):
272         nonlocal rng, a_max, local_prob, restart_prob,
    stagnation_limit, punish_scale, all_global_best
273 
274         if init_pop is None:
275             pop = np.array([random_individual_theta(rng) for _
     in range(pop_size)], dtype=np.float32)
276         else:
277             pop = [p.copy() for p in init_pop]
278             while len(pop) < pop_size:
279                 pop.append(random_individual_theta(rng))
280             pop = np.array(pop, dtype=np.float32)
281 
282         # 并行评估初始种群
283         results = parallel_simulate(pop, dt, t_after)
284         fitness = np.array([r.total for r in results], dtype=
    np.float32)
285         best_idx = int(np.argmax(fitness))
286         best_params = pop[best_idx].copy()
287         best_sim = results[best_idx]
288         best_val = best_sim.total
289         stagnation_counter = 0
290         all_global_best.append((best_val, best_params.copy(),
    best_sim))
291 
292         if verbose:
293             print(f"[WOA start] dt={dt:.4f} iters={iterations}
     pop={pop_size} initial_best={best_val:.4f}")
294 
295         for it in range(1, iterations+1):
296             a = a_max * (1 - (it/iterations)**0.5)
297 
298             # 随机重启
299             if rng.random() < restart_prob:
300                 replace_idx = rng.choice(pop_size, size=max(1,
```

```
 int(pop_size*0.08)), replace=False)
                for idx in replace_idx:
                    pop[idx] = random_individual_theta(rng)
                # 并行更新被替换的个体
                partial_results = parallel_simulate(pop[
replace_idx], dt, t_after)
                for i, idx in enumerate(replace_idx):
                    fitness[idx] = partial_results[i].total
                    if partial_results[i].total > best_val:
                        best_val = partial_results[i].total
                        best_params = pop[idx].copy()
                        best_sim = partial_results[i]
                        stagnation_counter = 0
                        all_global_best.append((best_val,
best_params.copy(), best_sim))

            # WOA 更新
            for i in range(pop_size):
                r1, r2 = rng.random(), rng.random()
                A = 2*a*r1 - a
                C = 2*r2
                p = rng.random()

                if p < 0.5:
                    if abs(A) < 1.0:
                        D = abs(C*best_params - pop[i])
                        pop[i] = best_params - A * D
                    else:
                        rand_idx = rng.integers(0, pop_size)
                        X_rand = pop[rand_idx]
                        D = abs(C*X_rand - pop[i])
                        pop[i] = X_rand - A * D
                else:
                    D = abs(best_params - pop[i])
                    b = 1
```

```
                    l = rng.uniform(-1,1)
                    pop[i] = D * np.exp(b*l) * np.cos(2*np.pi*l) + best_params

                if rng.random() < local_prob:
                    pop[i] = gaussian_mutation_theta(pop[i], rng=rng, rate=0.75, scale=0.12)
                pop[i] = clip_to_bounds_theta(pop[i])

            # 并行评估新种群
            results = parallel_simulate(pop, dt, t_after)
            fitness = np.array([r.total for r in results], dtype=np.float32)

            cur_best_idx = int(np.argmax(fitness))
            cur_best_val = fitness[cur_best_idx]

            if cur_best_val > best_val:
                best_params = pop[cur_best_idx].copy()
                best_sim = results[cur_best_idx]
                best_val = best_sim.total
                stagnation_counter = 0
                all_global_best.append((best_val, best_params.copy(), best_sim))
                if verbose:
                    print(f"[WOA] iter={it} new_best={best_val:.4f} params={np.round(best_params,3)}")
            else:
                stagnation_counter += 1

            # 提前终止检查
            if stagnation_counter >= stagnation_limit:
                if verbose:
                    print(f"Early termination at iteration {it} due to stagnation")
```

```
                break

        return best_params, best_sim, pop

    # -------- coarse stage --------
    best_coarse_params, best_coarse_sim, pop_coarse = run_woa(
        pop_size_coarse, iters_coarse, dt_coarse,
t_after_coarse)

    # -------- refine stage --------
    rng_local = np.random.default_rng(seed+1)
    init_pop_refine = [best_coarse_params.copy()]
    for _ in range(max(5, pop_size_refine-5)):
        pert = gaussian_mutation_theta(best_coarse_params.copy
(), rng=rng_local, rate=0.9, scale=0.08)
        init_pop_refine.append(pert)
    while len(init_pop_refine) < pop_size_refine:
        init_pop_refine.append(random_individual_theta(
rng_local))

    best_refine_params, best_refine_sim, _ = run_woa(
        pop_size_refine, iters_refine, dt_refine,
t_after_refine, init_pop=init_pop_refine)

    # 取所有最优结果
    max_total = max(g[0] for g in all_global_best)
    best_all = [(p, sim) for val, p, sim in all_global_best if
    val == max_total]

    if verbose:
        print(f"[All best solutions found: {len(best_all)}]
total={max_total:.4f}")

    return best_all
```

```python
# ==========================
# 输出 & 可视化（优化后）
# ==========================
def print_result_table_theta(sim: SimResult, grenade_id=1):
    u = sim.u_dir
    theta = np.arctan2(u[1], u[0])
    data = [
        ["无人机运动方向（水平平面 θ)", f"{theta:.3f} rad"],
        ["无人机运动速度 (m/s)", f"{sim.v_d:.3f}"],
        ["烟幕干扰弹编号", f"{grenade_id}"],
        ["烟幕干扰弹投放点 x (m)", f"{sim.P_drop[0]:.2f}"],
        ["烟幕干扰弹投放点 y (m)", f"{sim.P_drop[1]:.2f}"],
        ["烟幕干扰弹投放点 z (m)", f"{sim.P_drop[2]:.2f}"],
        ["烟幕干扰弹起爆点 x (m)", f"{sim.P_expl[0]:.2f}"],
        ["烟幕干扰弹起爆点 y (m)", f"{sim.P_expl[1]:.2f}"],
        ["烟幕干扰弹起爆点 z (m)", f"{sim.P_expl[2]:.2f}"],
        ["有效干扰时长 (s)", f"{sim.total:.3f}"],
    ]

    print("\n=== 优化结果一览（表格） ===")
    print(" | ".join([d[0] for d in data]))
    print(" | ".join([d[1] for d in data]))


def plot_heatmap_theta(sim: SimResult, dt=0.01, t_after=15.0):
    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.
   uint8)

    for j, tau in enumerate(times_rel):
        t_abs = sim.t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, sim.t_expl_abs, float(sim.
```

```
    P_expl[0]), float(sim.P_expl[1]), float(sim.P_expl[2]))
425         if C is None:
426             continue
427 
428         TM = TARGET_POINTS - M
429         CM = C - M
430         dot_product = np.einsum('ij,j->i', TM, CM, optimize=
    True)
431         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
432         lam = dot_product / TM_norm2
433         lam = np.clip(lam, 0.0, 1.0)
434         closest = M + (TM.T * lam).T
435         d2 = np.sum((C - closest)**2, axis=1)
436         obscured[:, j] = (d2 <= R*R).astype(np.uint8)
437 
438     plt.figure(figsize=(10, 6))
439     plt.imshow(obscured, aspect='auto', origin='lower', cmap='
    Greens')
440     plt.yticks([0, len(TARGET_POINTS)//2, len(TARGET_POINTS)
    -1], ["底部", "中部", "顶部"])
441     plt.xlabel("时间步")
442     plt.ylabel("目标点区域")
443     plt.title("遮蔽热力图")
444     plt.colorbar(label="是否被遮蔽")
445     plt.tight_layout()
446     plt.show()
447 
448 def plot_target_points():
449     fig = plt.figure(figsize=(10, 6))
450     ax = fig.add_subplot(111, projection='3d')
451     ax.scatter(TARGET_POINTS[:,0], TARGET_POINTS[:,1],
    TARGET_POINTS[:,2], s=5, c='r', alpha=0.6)
452     ax.set_xlabel('X (m)')
453     ax.set_ylabel('Y (m)')
```

```
    ax.set_zlabel('Z (m)')
    ax.set_title('目标采样点分布')
    plt.tight_layout()
    plt.show()

# ==========================
# 主程序（优化执行流程）
# ==========================
if __name__ == "__main__":
    print("优化开始，请等待...")
    start_time = time.time()

    # 运行优化算法（使用优化后的参数）
    all_best = enhanced_whale_optimization_two_stage_theta_all_best(
        pop_size_coarse=40,
        iters_coarse=100,
        dt_coarse=0.05,
        t_after_coarse=10.0,
        pop_size_refine=20,
        iters_refine=100,
        dt_refine=0.01,
        t_after_refine=15.0,
        verbose=True
    )

    # 输出结果
    print(f"\n优化完成，耗时: {time.time()-start_time:.2f}秒")
    for idx, (params, sim) in enumerate(all_best[:3]):  # 只显示前3个最优解
        print(f"\n--- 最优方案 {idx+1} ---")
        print_result_table_theta(sim, grenade_id=idx+1)
        if idx == 0:  # 只绘制第一个最优解的热图
            plot_heatmap_theta(sim)
```

Q5.py

```python
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from tqdm import tqdm
import time
from typing import Tuple
from mpl_toolkits.mplot3d import Axes3D
from concurrent.futures import ProcessPoolExecutor
import multiprocessing
from functools import lru_cache
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# ==========================
# 常量 & 场景参数
# ==========================
g = 9.81

# M1 与 FY1 初始状态
M0 = np.array([20000.0, 0.0, 2000.0], dtype=np.float32)
F0 = np.array([17800.0, 0.0, 1800.0], dtype=np.float32)

# —— 真目标（圆柱）——
target_center = np.array([0.0, 200.0, 0.0], dtype=np.float32)
target_radius = 7.0
target_height = 10.0

# —— 云团参数——
R = 10.0
sink_speed = 3.0
valid_after_explosion = 20.0

# M1 速度向量（固定）
sqrt101 = np.sqrt(101.0)
```

```
v_M_vec = 300.0 * np.array([-10.0/sqrt101, 0.0, -1.0/sqrt101],
    dtype=np.float32)
T_M_total = np.linalg.norm(M0 - np.array([0.0, 0.0, 0.0],
   dtype=np.float32)) / 300.0

# ==========================
# 运动学（带缓存优化）
# ==========================
@lru_cache(maxsize=1000)
def missile_position_abs(t_abs: float) -> np.ndarray:
    if t_abs >= T_M_total:
        return np.array([0.0, 0.0, 0.0], dtype=np.float32)
    return M0 + v_M_vec * t_abs

@lru_cache(maxsize=1000)
def drone_unit_and_speed_theta(theta: float, v_d: float):
    u_x = np.cos(theta)
    u_y = np.sin(theta)
    u = np.array([u_x, u_y, 0.0], dtype=np.float32)  # Z 坐标
   保持不变
    return u, v_d

@lru_cache(maxsize=1000)
def drone_position_abs_theta(t_abs: float, theta: float, v_d:
   float) -> np.ndarray:
    u, vd = drone_unit_and_speed_theta(theta, v_d)
    return F0 + u * vd * t_abs

@lru_cache(maxsize=1000)
def drop_point_theta(theta: float, v_d: float, t1: float) ->
   np.ndarray:
    P = drone_position_abs_theta(t1, theta, v_d)
    return np.array([P[0], P[1], F0[2]], dtype=np.float32)

@lru_cache(maxsize=1000)
```

```
64 def explosion_point_theta(theta: float, v_d: float, t1: float,
       tau_b: float) -> Tuple[float, np.ndarray]:
65     t_expl_abs = t1 + tau_b
66     P_h = drone_position_abs_theta(t_expl_abs, theta, v_d)
67     P_e = np.array([P_h[0], P_h[1], F0[2]], dtype=np.float32)
68     return t_expl_abs, P_e
69
70 @lru_cache(maxsize=1000)
71 def smoke_center_abs(t_abs: float, t_expl_abs: float, P_expl_x
      : float, P_expl_y: float, P_expl_z: float):
72     tau = t_abs - t_expl_abs
73     if tau < 0.0 or tau > valid_after_explosion:
74         return None
75     return np.array([P_expl_x, P_expl_y, P_expl_z - sink_speed
       * tau], dtype=np.float32)
76
77 # ==========================
78 # 目标几何采样（带缓存和单精度优化）
79 # ==========================
80 _TARGET_POINTS_CACHE = None
81
82 def sample_cylinder_surface(radius, height,
      num_points_per_circle=360, num_height_samples=50):
83     global _TARGET_POINTS_CACHE
84     if _TARGET_POINTS_CACHE is not None:
85         return _TARGET_POINTS_CACHE
86
87     pts = []
88     angles = np.linspace(0, 2*np.pi, num_points_per_circle,
      endpoint=False)
89     heights = np.linspace(0, height, num_height_samples)
90
91     # 圆柱侧面采样
92     for z in heights:
93         for ang in angles:
```

```
            x = target_center[0] + radius * np.cos(ang)
            y = target_center[1] + radius * np.sin(ang)
            pts.append([x, y, z])

    # 圆柱顶部边缘采样
    for ang in angles:
        x = target_center[0] + radius * np.cos(ang)
        y = target_center[1] + radius * np.sin(ang)
        pts.append([x, y, height])

    # 顶部中心点
    pts.append([target_center[0], target_center[1], height])

    _TARGET_POINTS_CACHE = np.array(pts, dtype=np.float32)
    return _TARGET_POINTS_CACHE

# 预计算目标点
TARGET_POINTS = sample_cylinder_surface(target_radius,
   target_height)

# ==========================
# 遮蔽仿真（并行优化）
# ==========================
@dataclass
class SimResult:
    intervals: list
    total: float
    t_expl_abs: float
    P_drop: np.ndarray
    P_expl: np.ndarray
    u_dir: np.ndarray
    v_d: float

_sim_cache = {}

```

```
def _params_key(theta, v_d, t1, tau_b, dt, t_after):
    return (round(float(theta), 4), round(float(v_d), 3),
            round(float(t1), 3), round(float(tau_b), 3),
            round(float(dt), 4), round(float(t_after), 3))

def _simulate_single(params, dt, t_after):
    """单个参数的仿真计算"""
    theta, v_d, t1, tau_b = params
    key = _params_key(theta, v_d, t1, tau_b, dt, t_after)
    if key in _sim_cache:
        return _sim_cache[key]

    P_drop = drop_point_theta(theta, v_d, t1)
    t_expl_abs, P_expl = explosion_point_theta(theta, v_d, t1,
    tau_b)
    u_dir, _ = drone_unit_and_speed_theta(theta, v_d)

    if P_expl[2] < 0.0:
        res = SimResult([], 0.0, t_expl_abs, P_drop, P_expl,
   u_dir, v_d)
        _sim_cache[key] = res
        return res

    times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
    num_times = len(times_rel)
    num_targets = len(TARGET_POINTS)
    obscured = np.zeros((num_targets, num_times), dtype=np.
   uint8)

    for j, tau in enumerate(times_rel):
        t_abs = t_expl_abs + tau
        M = missile_position_abs(t_abs)
        C = smoke_center_abs(t_abs, t_expl_abs, float(P_expl
   [0]), float(P_expl[1]), float(P_expl[2]))
        if C is None:
```

```
159             continue
160
161         # 向量化计算遮蔽
162         TM = TARGET_POINTS - M
163         CM = C - M
164         dot_product = np.einsum('ij,j->i', TM, CM, optimize=
    True)
165         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
166         lam = dot_product / TM_norm2
167         lam = np.clip(lam, 0.0, 1.0)
168         closest = M + (TM.T * lam).T  # 高效广播计算
169         d2 = np.sum((C - closest)**2, axis=1)
170         obscured[:, j] = (d2 <= R*R).astype(np.uint8)
171
172     full_mask = np.all(obscured == 1, axis=0)
173     idx = np.where(full_mask)[0]
174     intervals = []
175     total = 0.0
176
177     if idx.size:
178         runs = np.split(times_rel[idx], np.where(np.diff(idx)
    != 1)[0]+1)
179         total = sum(r[-1] - r[0] for r in runs)
180         if len(runs) > 0:
181             intervals = [(r[0], r[-1]) for r in runs]
182
183     res = SimResult(intervals, float(total), float(t_expl_abs)
    , P_drop, P_expl, u_dir, v_d)
184     _sim_cache[key] = res
185     return res
186
187 def simulate_duration_theta(theta: float, v_d: float, t1:
    float, tau_b: float,
188                             dt: float = 0.01, t_after: float =
```

```
    15.0,
                                want_intervals=True) -> SimResult:
    return _simulate_single((theta, v_d, t1, tau_b), dt,
   t_after)

# 并行计算包装函数（解决pickle问题）
def _simulate_wrapper(args):
    """用于并行计算的包装函数"""
    params, dt, t_after = args
    return _simulate_single(params, dt, t_after)

def parallel_simulate(params_list, dt, t_after):
    """并行计算多个参数的仿真结果"""
    # 准备参数列表
    args_list = [(p, dt, t_after) for p in params_list]

    # 根据CPU核心数确定工作进程数
    num_workers = min(multiprocessing.cpu_count(), len(
   params_list))

    with ProcessPoolExecutor(max_workers=num_workers) as
   executor:
        results = list(tqdm(executor.map(_simulate_wrapper,
   args_list),
                            total=len(args_list),
                            desc="并行计算进度"))
    return results

# =========================
# WOA 边界与个体生成
# =========================
@dataclass
class BoundsTheta:
    theta: tuple = (0.0, 2*np.pi)
    vd: tuple = (70.0, 140.0)
```

```
    t1: tuple = (0.0, 40.0)
    tb: tuple = (0.0, 40.0)

BOUNDS_THETA = BoundsTheta()

def random_individual_theta(rng: np.random.Generator) -> np.ndarray:
    theta = rng.uniform(*BOUNDS_THETA.theta)
    vd = rng.uniform(*BOUNDS_THETA.vd)
    t1 = rng.uniform(*BOUNDS_THETA.t1)
    tb = rng.uniform(*BOUNDS_THETA.tb)
    return np.array([theta, vd, t1, tb], dtype=np.float32)

def clip_to_bounds_theta(x: np.ndarray) -> np.ndarray:
    x[0] = np.clip(x[0], *BOUNDS_THETA.theta)
    x[1] = np.clip(x[1], *BOUNDS_THETA.vd)
    x[2] = np.clip(x[2], *BOUNDS_THETA.t1)
    x[3] = np.clip(x[3], *BOUNDS_THETA.tb)
    return x

def gaussian_mutation_theta(x, rng=None, rate=0.8, scale=0.1):
    ranges = np.array([BOUNDS_THETA.theta[1]-BOUNDS_THETA.theta[0],
                       BOUNDS_THETA.vd[1]-BOUNDS_THETA.vd[0],
                       BOUNDS_THETA.t1[1]-BOUNDS_THETA.t1[0],
                       BOUNDS_THETA.tb[1]-BOUNDS_THETA.tb[0]],
    dtype=np.float32)
    for i in range(len(x)):
        if rng.random() < rate:
            x[i] += rng.normal(0.0, scale * ranges[i])
    return clip_to_bounds_theta(x)

# ==========================
# 两阶段鲸鱼优化算法（并行优化）
# ==========================
```

```python
def enhanced_whale_optimization_two_stage_theta_all_best(
        pop_size_coarse=40,  # 减少粗搜索种群大小
        iters_coarse=30,     # 减少粗搜索迭代次数
        dt_coarse=0.05,      # 增大粗搜索时间步长
        t_after_coarse=10.0, # 缩短粗搜索时间范围
        pop_size_refine=20,  # 减少精搜索种群大小
        iters_refine=30,     # 减少精搜索迭代次数
        dt_refine=0.01,      # 适度增大精搜索时间步长
        t_after_refine=15.0, # 适度缩短精搜索时间范围
        a_max=2.5,
        local_prob=0.35,
        restart_prob=0.04,
        stagnation_limit=5,  # 更早终止
        punish_scale=0.25,
        seed=2025,
        verbose=True):

    rng = np.random.default_rng(seed)
    all_global_best = []

    def run_woa(pop_size, iterations, dt, t_after, init_pop=None):
        nonlocal rng, a_max, local_prob, restart_prob, stagnation_limit, punish_scale, all_global_best

        if init_pop is None:
            pop = np.array([random_individual_theta(rng) for _ in range(pop_size)], dtype=np.float32)
        else:
            pop = [p.copy() for p in init_pop]
            while len(pop) < pop_size:
                pop.append(random_individual_theta(rng))
            pop = np.array(pop, dtype=np.float32)

        # 并行评估初始种群
```

```
283         results = parallel_simulate(pop, dt, t_after)
284         fitness = np.array([r.total for r in results], dtype=
    np.float32)
285         best_idx = int(np.argmax(fitness))
286         best_params = pop[best_idx].copy()
287         best_sim = results[best_idx]
288         best_val = best_sim.total
289         stagnation_counter = 0
290         all_global_best.append((best_val, best_params.copy(),
    best_sim))
291 
292         if verbose:
293             print(f"[WOA start] dt={dt:.4f} iters={iterations}
     pop={pop_size} initial_best={best_val:.4f}")
294 
295         for it in range(1, iterations+1):
296             a = a_max * (1 - (it/iterations)**0.5)
297 
298             # 随机重启
299             if rng.random() < restart_prob:
300                 replace_idx = rng.choice(pop_size, size=max(1,
     int(pop_size*0.08)), replace=False)
301                 for idx in replace_idx:
302                     pop[idx] = random_individual_theta(rng)
303                 # 并行更新被替换的个体
304                 partial_results = parallel_simulate(pop[
    replace_idx], dt, t_after)
305                 for i, idx in enumerate(replace_idx):
306                     fitness[idx] = partial_results[i].total
307                     if partial_results[i].total > best_val:
308                         best_val = partial_results[i].total
309                         best_params = pop[idx].copy()
310                         best_sim = partial_results[i]
311                         stagnation_counter = 0
312                         all_global_best.append((best_val,
```

```
    best_params.copy(), best_sim))
313
314             # WOA 更新
315             for i in range(pop_size):
316                 r1, r2 = rng.random(), rng.random()
317                 A = 2*a*r1 - a
318                 C = 2*r2
319                 p = rng.random()
320
321                 if p < 0.5:
322                     if abs(A) < 1.0:
323                         D = abs(C*best_params - pop[i])
324                         pop[i] = best_params - A * D
325                     else:
326                         rand_idx = rng.integers(0, pop_size)
327                         X_rand = pop[rand_idx]
328                         D = abs(C*X_rand - pop[i])
329                         pop[i] = X_rand - A * D
330                 else:
331                     D = abs(best_params - pop[i])
332                     b = 1
333                     l = rng.uniform(-1,1)
334                     pop[i] = D * np.exp(b*l) * np.cos(2*np.pi*
    l) + best_params
335
336                 if rng.random() < local_prob:
337                     pop[i] = gaussian_mutation_theta(pop[i],
    rng=rng, rate=0.75, scale=0.12)
338                 pop[i] = clip_to_bounds_theta(pop[i])
339
340             # 并行评估新种群
341             results = parallel_simulate(pop, dt, t_after)
342             fitness = np.array([r.total for r in results],
    dtype=np.float32)
343
```

```
            cur_best_idx = int(np.argmax(fitness))
            cur_best_val = fitness[cur_best_idx]

            if cur_best_val > best_val:
                best_params = pop[cur_best_idx].copy()
                best_sim = results[cur_best_idx]
                best_val = best_sim.total
                stagnation_counter = 0
                all_global_best.append((best_val, best_params.copy(), best_sim))
                if verbose:
                    print(f"[WOA] iter={it} new_best={best_val:.4f} params={np.round(best_params,3)}")
            else:
                stagnation_counter += 1

            # 提前终止检查
            if stagnation_counter >= stagnation_limit:
                if verbose:
                    print(f"Early termination at iteration {it} due to stagnation")
                break

        return best_params, best_sim, pop

    # -------- coarse stage --------
    best_coarse_params, best_coarse_sim, pop_coarse = run_woa(
        pop_size_coarse, iters_coarse, dt_coarse, t_after_coarse)

    # -------- refine stage --------
    rng_local = np.random.default_rng(seed+1)
    init_pop_refine = [best_coarse_params.copy()]
    for _ in range(max(5, pop_size_refine-5)):
        pert = gaussian_mutation_theta(best_coarse_params.copy
```

```
    (), rng=rng_local, rate=0.9, scale=0.08)
375         init_pop_refine.append(pert)
376     while len(init_pop_refine) < pop_size_refine:
377         init_pop_refine.append(random_individual_theta(
    rng_local))
378 
379     best_refine_params, best_refine_sim, _ = run_woa(
380         pop_size_refine, iters_refine, dt_refine,
    t_after_refine, init_pop=init_pop_refine)
381 
382     # 取所有最优结果
383     max_total = max(g[0] for g in all_global_best)
384     best_all = [(p, sim) for val, p, sim in all_global_best if
     val == max_total]
385 
386     if verbose:
387         print(f"[All best solutions found: {len(best_all)}]
    total={max_total:.4f}")
388 
389     return best_all
390 
391 # ==========================
392 # 输出 & 可视化（优化后）
393 # ==========================
394 def print_result_table_theta(sim: SimResult, grenade_id=1):
395     u = sim.u_dir
396     theta = np.arctan2(u[1], u[0])
397     data = [
398         ["无人机运动方向（水平平面 θ)", f"{theta:.3f} rad"],
399         ["无人机运动速度 (m/s)", f"{sim.v_d:.3f}"],
400         ["烟幕干扰弹编号", f"{grenade_id}"],
401         ["烟幕干扰弹投放点 x (m)", f"{sim.P_drop[0]:.2f}"],
402         ["烟幕干扰弹投放点 y (m)", f"{sim.P_drop[1]:.2f}"],
403         ["烟幕干扰弹投放点 z (m)", f"{sim.P_drop[2]:.2f}"],
404         ["烟幕干扰弹起爆点 x (m)", f"{sim.P_expl[0]:.2f}"],
```

```
405         ["烟幕干扰弹起爆点 y (m)", f"{sim.P_expl[1]:.2f}"],
406         ["烟幕干扰弹起爆点 z (m)", f"{sim.P_expl[2]:.2f}"],
407         ["有效干扰时长 (s)", f"{sim.total:.3f}"],
408     ]
409 
410     print("\n=== 优化结果一览（表格） ===")
411     print(" | ".join([d[0] for d in data]))
412     print(" | ".join([d[1] for d in data]))
413 
414 
415 def plot_heatmap_theta(sim: SimResult, dt=0.01, t_after=15.0):
416     times_rel = np.arange(0.0, t_after, dt, dtype=np.float32)
417     num_times = len(times_rel)
418     num_targets = len(TARGET_POINTS)
419     obscured = np.zeros((num_targets, num_times), dtype=np.
    uint8)
420 
421     for j, tau in enumerate(times_rel):
422         t_abs = sim.t_expl_abs + tau
423         M = missile_position_abs(t_abs)
424         C = smoke_center_abs(t_abs, sim.t_expl_abs, float(sim.
    P_expl[0]), float(sim.P_expl[1]), float(sim.P_expl[2]))
425         if C is None:
426             continue
427 
428         TM = TARGET_POINTS - M
429         CM = C - M
430         dot_product = np.einsum('ij,j->i', TM, CM, optimize=
    True)
431         TM_norm2 = np.einsum('ij,ij->i', TM, TM, optimize=True
    )
432         lam = dot_product / TM_norm2
433         lam = np.clip(lam, 0.0, 1.0)
434         closest = M + (TM.T * lam).T
435         d2 = np.sum((C - closest)**2, axis=1)
```

```
436             obscured[:, j] = (d2 <= R*R).astype(np.uint8)
437 
438         plt.figure(figsize=(10, 6))
439         plt.imshow(obscured, aspect='auto', origin='lower', cmap='
    Greens')
440         plt.yticks([0, len(TARGET_POINTS)//2, len(TARGET_POINTS)
    -1], ["底部", "中部", "顶部"])
441         plt.xlabel("时间步")
442         plt.ylabel("目标点区域")
443         plt.title("遮蔽热力图")
444         plt.colorbar(label="是否被遮蔽")
445         plt.tight_layout()
446         plt.show()
447 
448 def plot_target_points():
449         fig = plt.figure(figsize=(10, 6))
450         ax = fig.add_subplot(111, projection='3d')
451         ax.scatter(TARGET_POINTS[:,0], TARGET_POINTS[:,1],
    TARGET_POINTS[:,2], s=5, c='r', alpha=0.6)
452         ax.set_xlabel('X (m)')
453         ax.set_ylabel('Y (m)')
454         ax.set_zlabel('Z (m)')
455         ax.set_title('目标采样点分布')
456         plt.tight_layout()
457         plt.show()
458 
459 # ==========================
460 # 主程序（优化执行流程）
461 # ==========================
462 if __name__ == "__main__":
463         print("优化开始，请等待...")
464         start_time = time.time()
465 
466         # 运行优化算法（使用优化后的参数）
467         all_best =
```

```
    enhanced_whale_optimization_two_stage_theta_all_best(
468         pop_size_coarse=40,
469         iters_coarse=100,
470         dt_coarse=0.05,
471         t_after_coarse=10.0,
472         pop_size_refine=20,
473         iters_refine=100,
474         dt_refine=0.01,
475         t_after_refine=15.0,
476         verbose=True
477     )
478 
479     # 输出结果
480     print(f"\n优化完成，耗时: {time.time()-start_time:.2f}秒")
481     for idx, (params, sim) in enumerate(all_best[:3]):  # 只显
    示前3个最优解
482         print(f"\n--- 最优方案 {idx+1} ---")
483         print_result_table_theta(sim, grenade_id=idx+1)
484         if idx == 0:  # 只绘制第一个最优解的热图
485             plot_heatmap_theta(sim)
```